

Data Version Control

TA. Đăng Nhã
STA. Anh Dương

Outline

Theory Perspective

1. Recap AI, MLOPs
2. Data versioning?
3. ML – MLOPs pipeline
4. What is DVC?

Section 1

Case Study Implementation

1. Initialize Problem
2. DVC and Git setup
3. Step by step versioning data with DVC

Section 2

Automate Pipeline with DVC & The Idea of Versioning

1. Software & Data Experiment Workflow Diagram

Section 3

[Quiz&Feedbacks](#)

Theory Perspective



Theory Perspective

❖ AI in Software

```
def analyze_sentiment(review):  
    positive_keywords = ["great", "love", "excellent"]  
    negative_keywords = ["bad", "hate", "terrible"]  
    score = 0  
    for word in review.split():  
        if word.lower() in positive_keywords:  
            score += 1  
        elif word.lower() in negative_keywords:  
            score -= 1  
    if score > 0:  
        return "Positive"  
    elif score < 0:  
        return "Negative"  
    else:  
        return "Neutral"
```

Traditional Programming

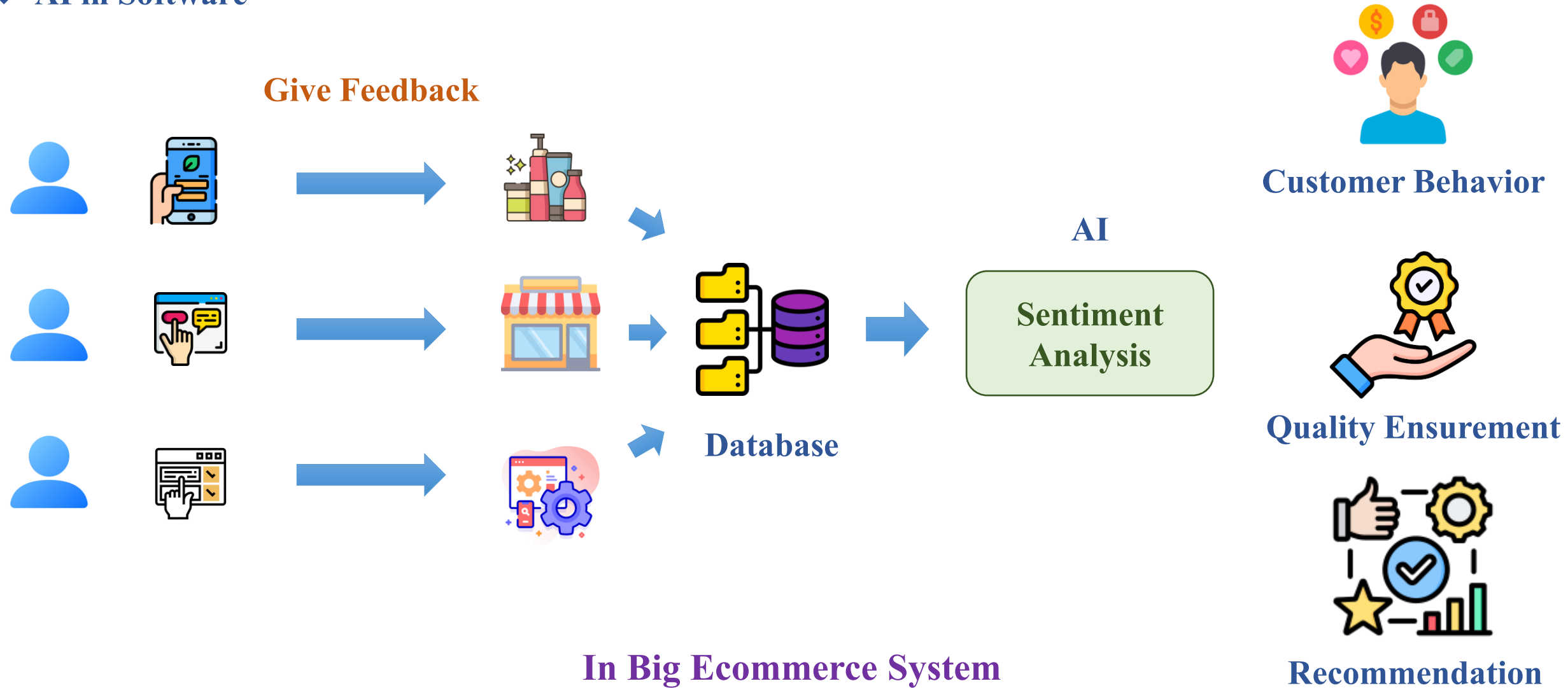
```
from transformers import pipeline  
  
sentiment_analyzer = pipeline("sentiment-analysis")  
  
def analyze_sentiment_ai(review):  
    result = sentiment_analyzer(review)[0]  
    return result['label']  
# Outputs 'POSITIVE' or 'NEGATIVE' with confidence score
```

AI-based

Sentiment Analysis

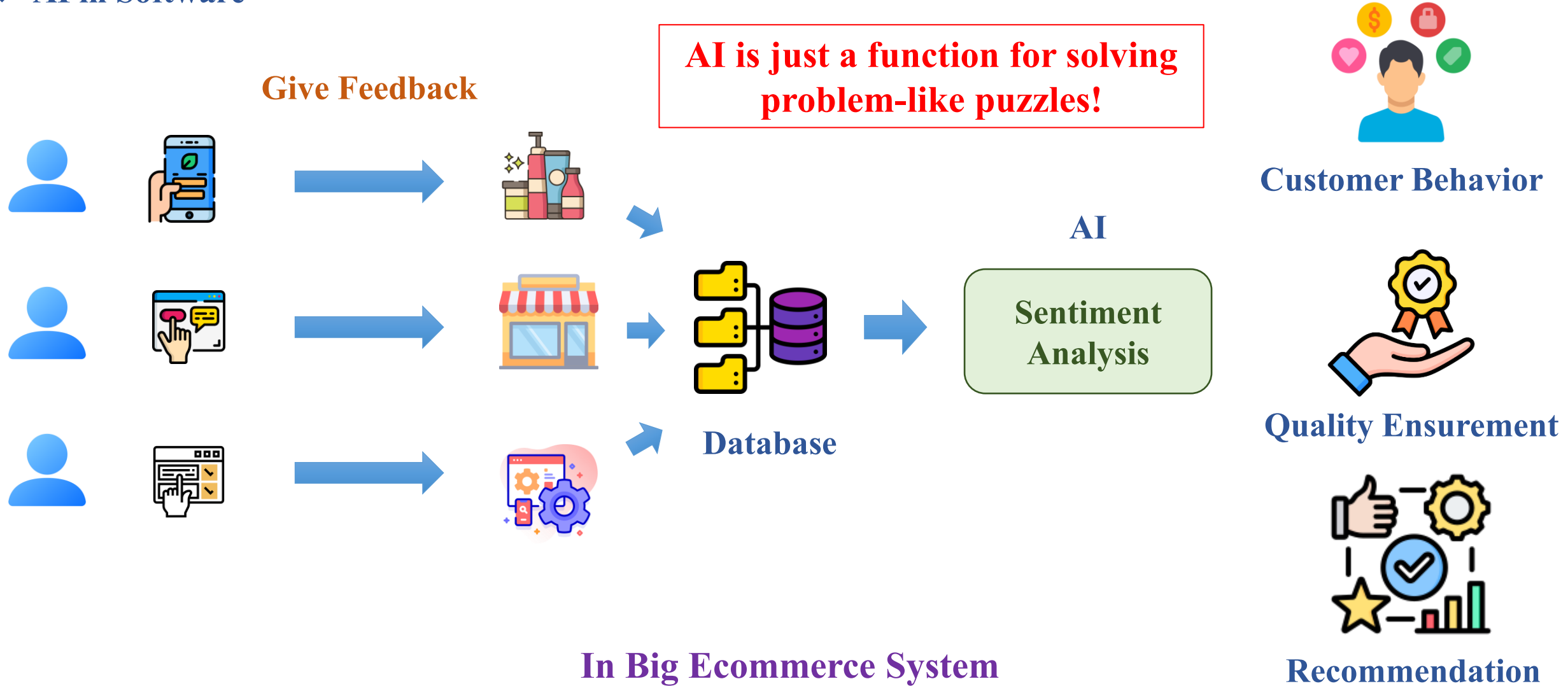
Theory Perspective

❖ AI in Software



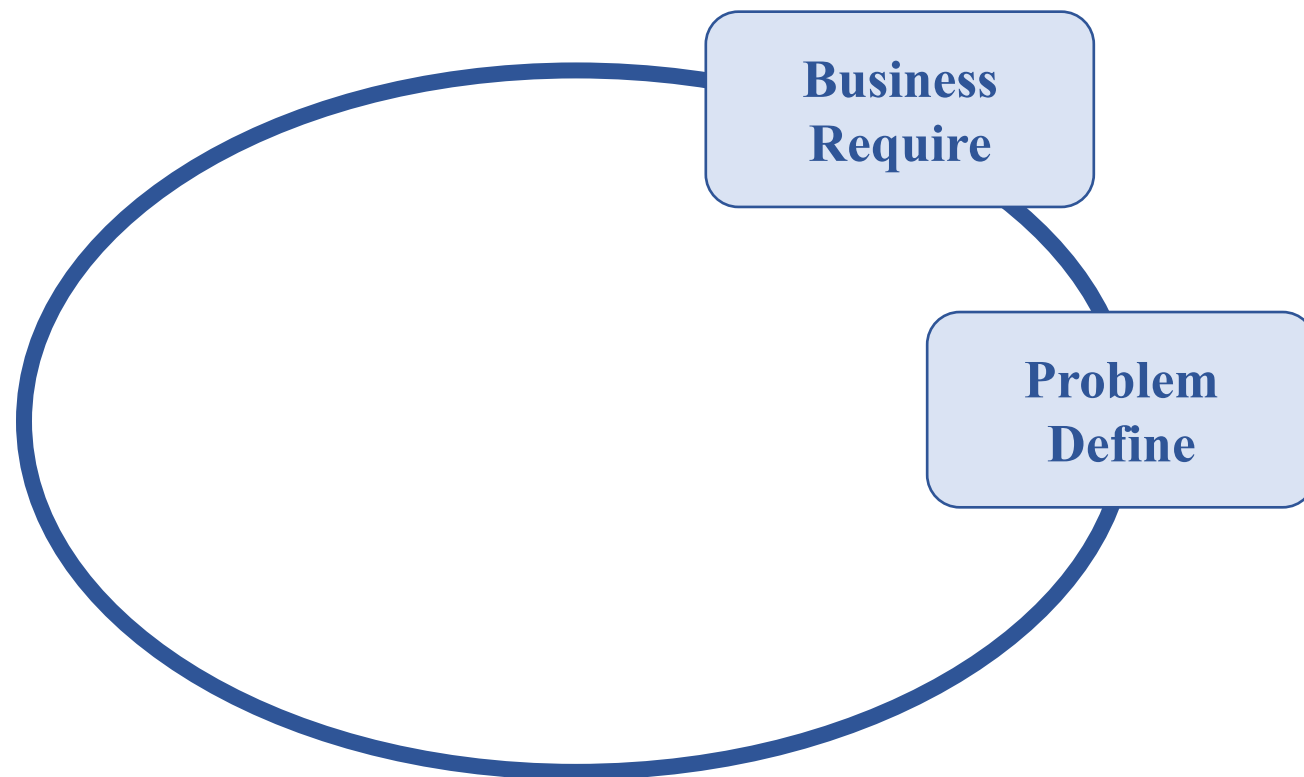
Theory Perspective

❖ AI in Software



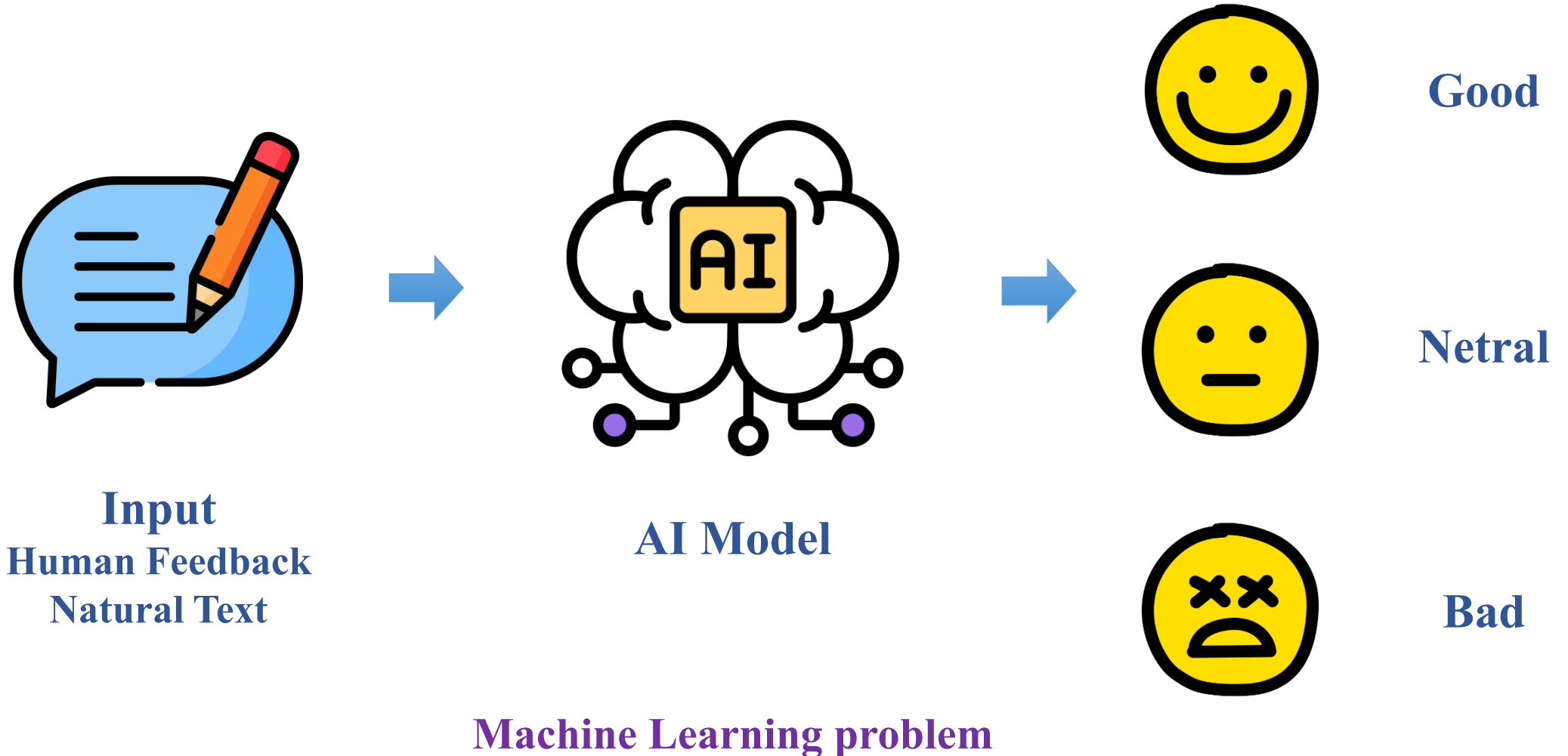


❖ ML Life Cycle



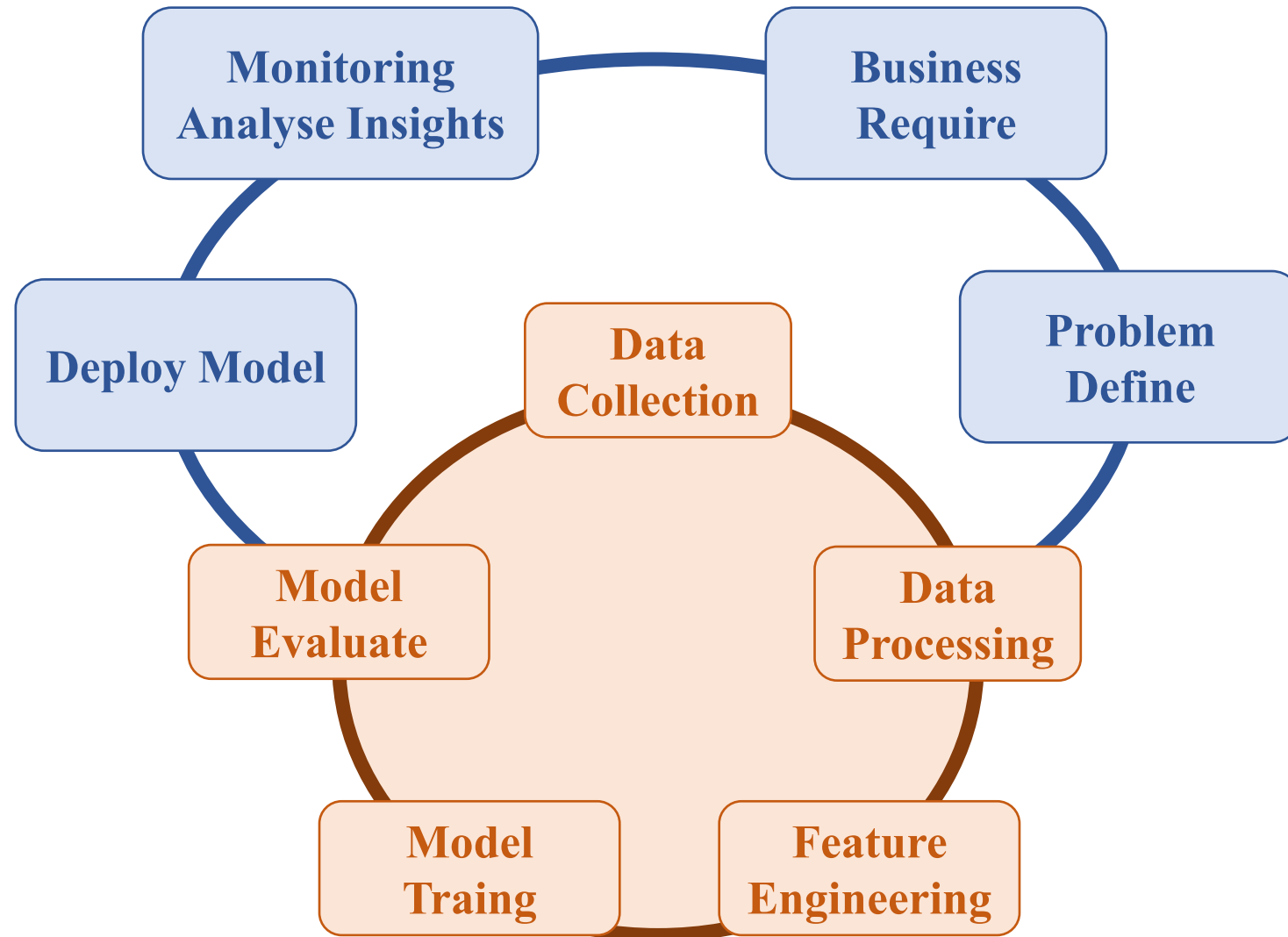
Theory Perspective

❖ Problem Define – Sentiment Analysis



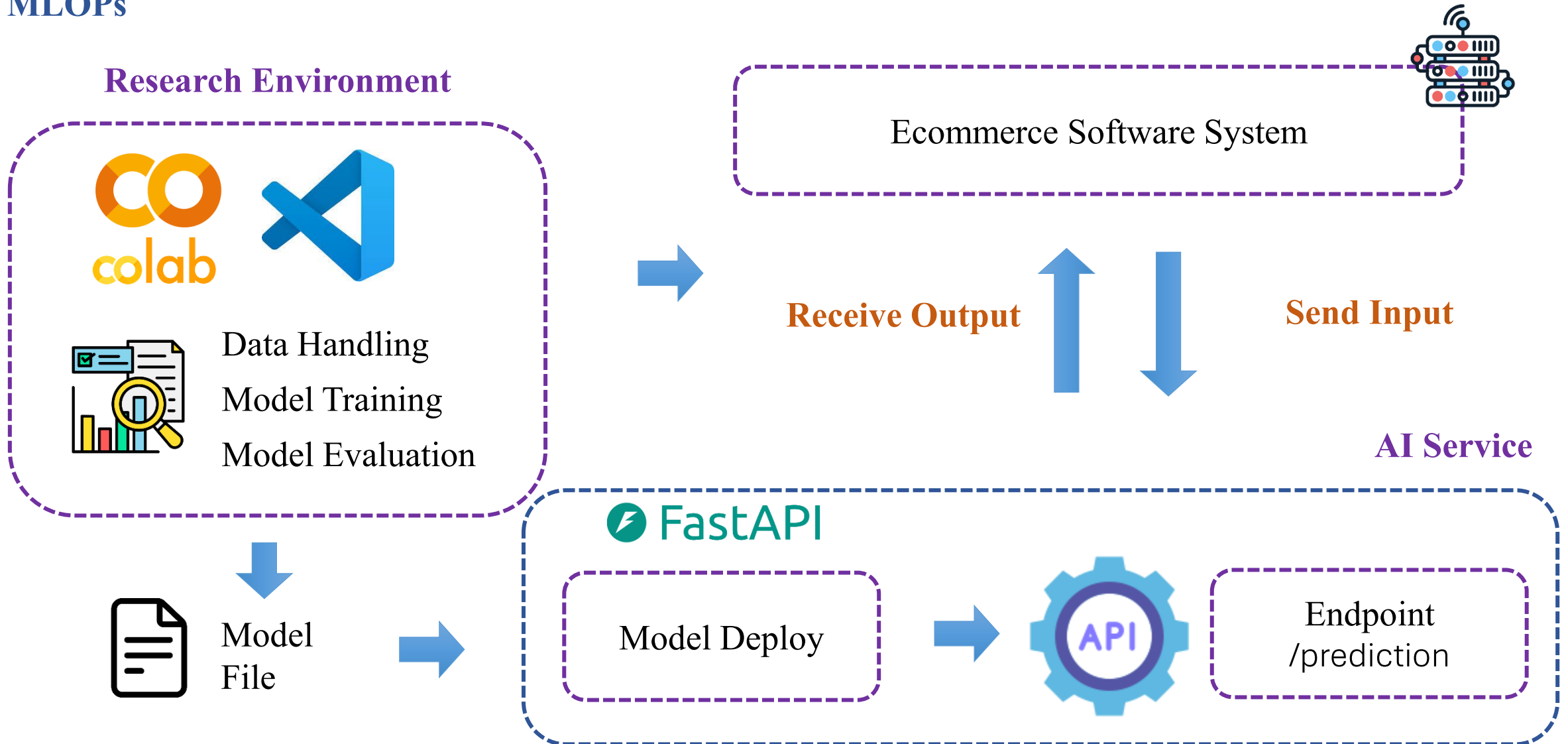
Theory Perspective

❖ ML Life Cycle



Theory Perspective

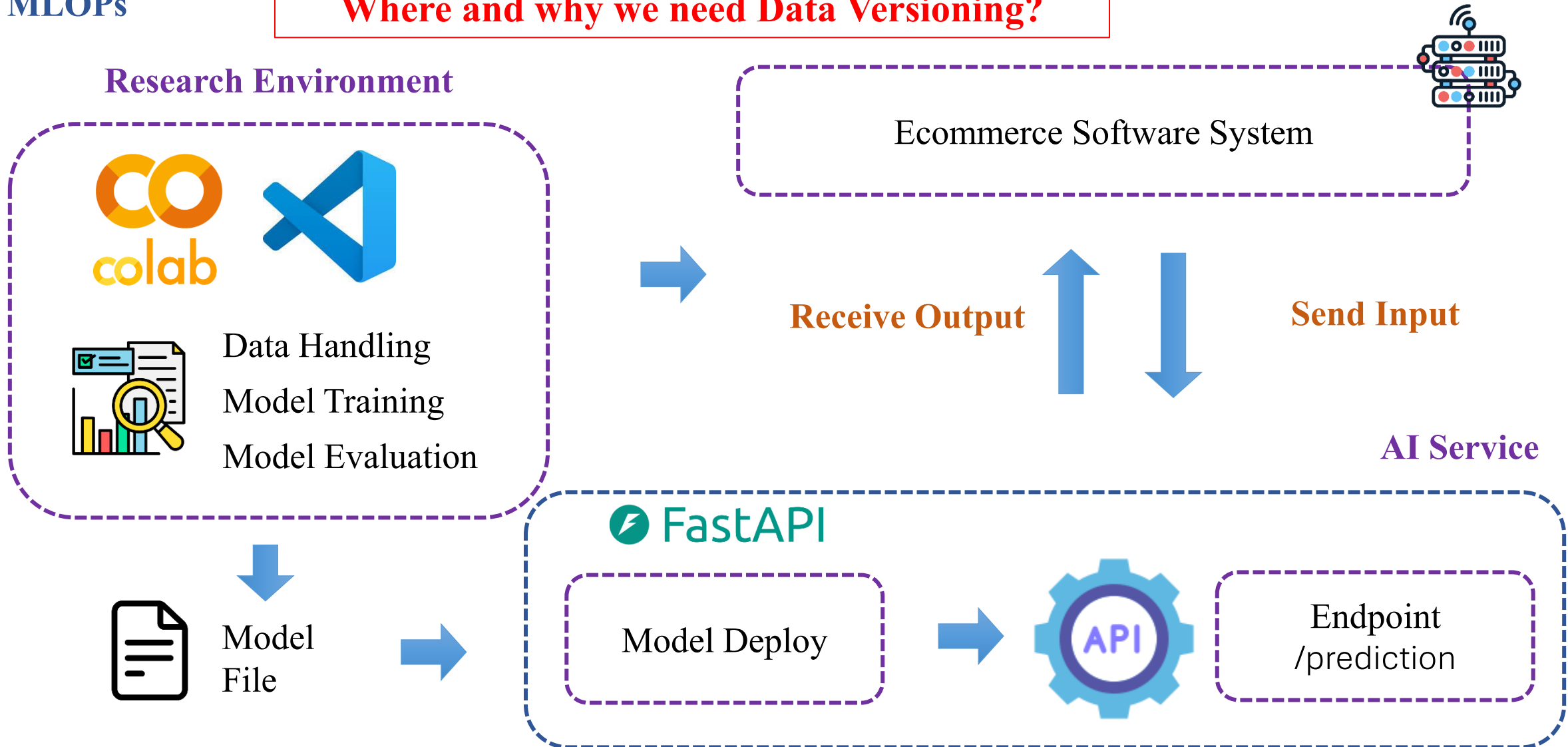
❖ MLOPs



Theory Perspective

❖ MLOPs

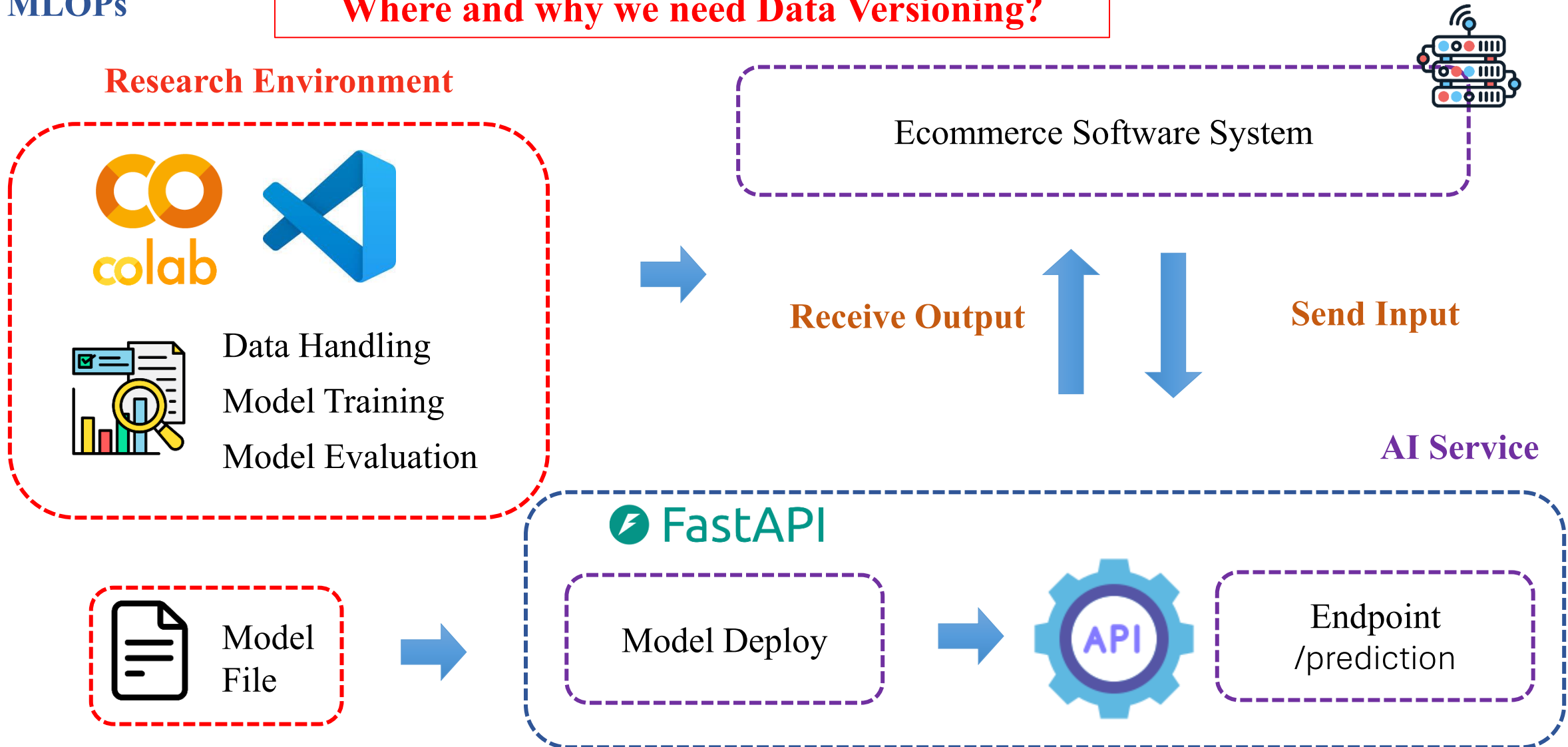
Where and why we need Data Versioning?



Theory Perspective

❖ MLOPs

Where and why we need Data Versioning?



Theory Perspective

❖ My problem on Data Version

IMA: An Imputation-based Mixup Augmentation Using Self-Supervised Learning for Time Series Data

Nha Dang Nguyen¹, Dang Hai Nguyen², Khoa Tho Anh Nguyen^{3*}

¹Vietnam - Korea University of Information and Communication Technology, Da Nang City, Vietnam

²VNU University of Engineering and Technology, Ha Noi City, Vietnam

³Vietnamese – German University, Ho Chi Minh City, Vietnam

nhand.21it@vku.udn.vn, 24025015@vnu.edu.vn, 30421001@student.vgu.edu.vn

Abstract

Data augmentation plays a crucial role in enhancing model performance across various AI fields by introducing variability while maintaining the underlying temporal patterns. However, in the context of long sequence time series data, where maintaining temporal consistency is critical, there are fewer augmentation strategies compared to fields such as image or text, with advanced techniques like Mixup rarely being used. In this work, we propose a new approach, Imputation-based Mixup Augmentation (IMA), which combines Imputed-data Augmentation with Mixup Augmentation to bolster model generalization and improve forecasting performance. We evaluate the effectiveness of this method across several forecasting models, including DLinear (MLP), TimesNet (CNN), and iTransformer (Transformer), these models represent some of the most recent advances in long sequence time series forecasting. Our experiments, conducted on three datasets (ETT-small, Illness, Exchange Rate) from various domains and compared against eight other augmentation techniques, demonstrate that IMA consistently enhances performance, achieving 22 improvements out of 24 instances, with 10 of those being the best performances, particularly with iTransformer imputation in ETT dataset. The GitHub repository is available at: <https://github.com/dangnha/IMA>.

Introduction

Time series forecasting has gained significant attention, especially long sequence time-series forecasting (LSTF), due to its wide range of applications across domains such as finance, healthcare, energy management, meteorology, and

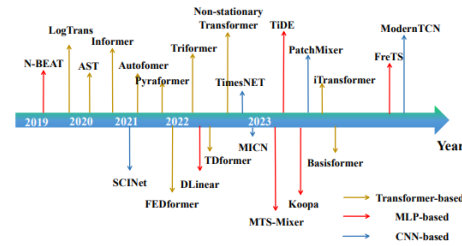


Figure 1: Key Milestones in Time Series Forecasting: Evolution of MLP-based, CNN-based, and Transformer-based Models (Liu and Wang 2024)

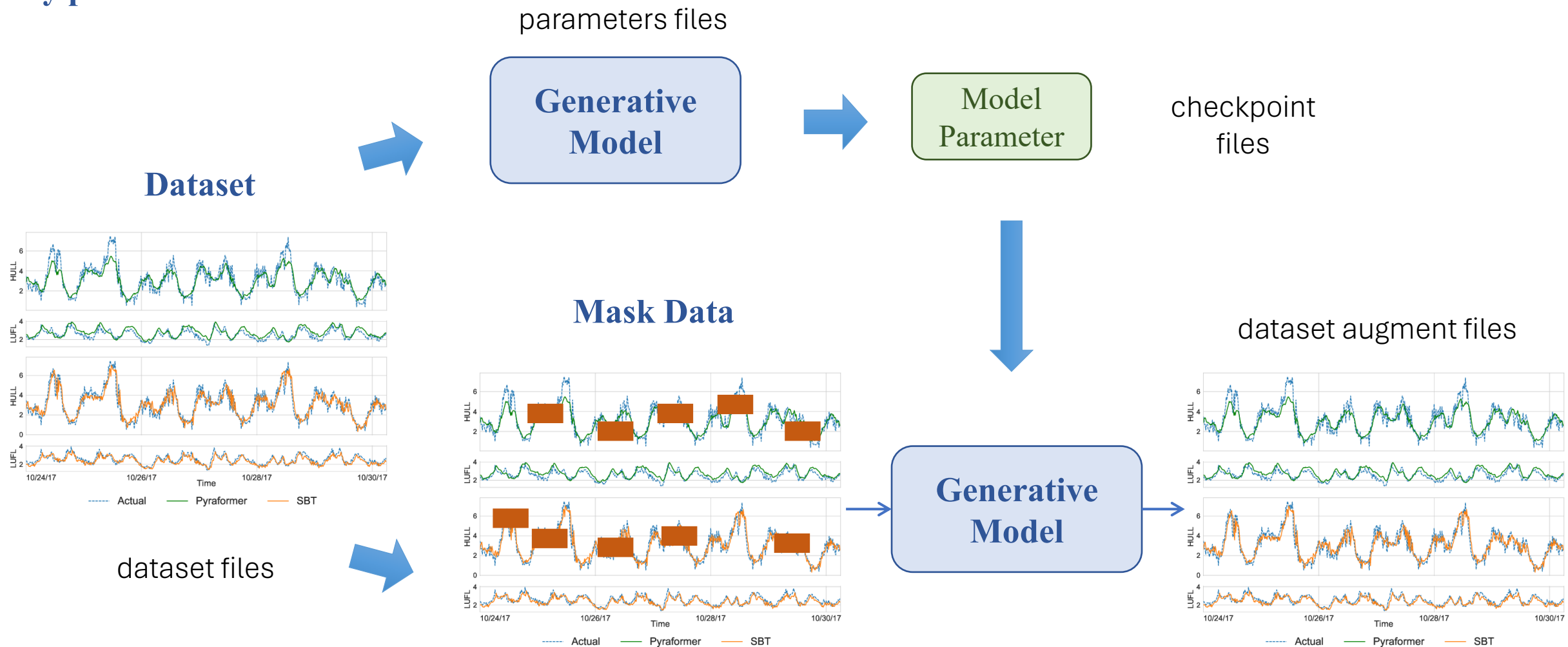
as Multi-Layer Perceptrons (MLPs) (Lai et al. 2018; Salinas et al. 2020; Oreshkin et al. 2020; Zeng et al. 2023), Convolutional Neural Networks (CNNs) (Huang et al. 2019; Zhou et al. 2021b; Li et al. 2021; LIU et al. 2022; Wu et al. 2023), and Transformer-based architectures (Wu et al. 2021; Zhang and Yan 2023; Tang and Cai 2023; Liu et al. 2024) have demonstrated state-of-the-art performance by improving temporal dependency modeling and scalability for diverse LSTF tasks throughout recent year (showed in figure 1). These advancements have created robust pipelines, emphasizing the importance of data preprocessing, feature ex-

Tons of Experiments

Model	DLinear		TimesNet		iTransformer		DLinear		TimesNet		iTransformer	
Metric	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
Dataset	ETTth1											
Baseline	0.445051	0.440448	0.458988	0.45493	0.447285	0.440212	0.479687	0.477797	0.406138	0.413908	0.4578	0.398759
Jitter	0.00E+00	1.00E-06	-1.37E-03	-7.73E-04	-1.10E-05	7.06E-03	3.00E-05	2.20E-05	3.99E-03	3.63E-03	-7.84E-02	-1.13E-04
Hflip	-5.73E-04	-2.07E-04	7.94E-03	2.84E-03	-5.47E-03	-4.07E-03	1.24E-03	1.22E-03	-1.63E-02	-7.31E-03	-5.92E-02	-2.66E-01
Vflip	4.41E-03	4.89E-03	-6.30E-03	-2.93E-03	-5.86E-03	-4.96E-03	7.55E-03	6.83E-03	3.36E-02	1.29E-02	-7.43E-02	-7.35E-04
Scaling	7.93E-04	1.14E-03	-1.34E-03	-6.42E-04	-2.98E-01	-2.93E-01	-6.82E-04	1.97E-04	2.07E-02	1.29E-02	-7.80E-02	-2.83E-04
Win.warp	1.81E-02	1.81E-02	3.71E-02	2.06E-02	2.21E-02	1.30E-02	-6.41E-04	9.71E-04	-1.16E-04	3.98E-03	-7.46E-02	2.40E-03
Win.slide	6.66E-02	4.17E-02	1.65E-02	1.50E-02	2.79E-02	1.76E-02	7.63E-03	7.55E-03	1.48E-02	1.23E-02	-6.57E-02	9.67E-03
Permu	-1.66E-04	4.87E-04	-3.60E-03	1.44E-03	1.05E-02	1.76E-02	8.17E-03	6.42E-03	1.21E-02	5.43E-03	-7.62E-02	1.34E-04
Mixup	2.10E-04	1.92E-04	7.78E-04	-4.23E-04	-1.58E-03	-1.38E-03	-1.01E-02	-5.84E-03	-2.67E-03	-1.06E-03	-7.88E-02	-3.87E-04
TS.IA	-4.54E-03	-3.23E-03	-1.41E-04	-1.48E-03	-1.78E-03	-1.89E-03	-1.87E-02	-1.20E-02	1.17E-03	1.02E-04	-7.74E-02	-2.00E-03
iT.IA	-5.73E-03	-4.84E-03	-7.39E-03	-7.65E-03	-1.33E-03	-1.96E-03	-1.78E-02	-1.16E-02	-7.39E-03	-3.62E-03	-7.49E-02	-3.09E-04
TS.IMA	-6.31E-03	-6.11E-03	-5.13E-03	-6.76E-03	-3.68E-03	-3.60E-03	-2.15E-02	-1.30E-02	-1.28E-02	-7.06E-03	-7.65E-02	-1.84E-03
iT.IMA	-6.38E-03	-5.85E-03	-1.54E-02	-1.18E-02	-3.14E-03	-3.98E-03	-1.25E-02	-8.32E-03	-1.18E-02	-6.75E-03	-7.52E-02	-2.02E-04
Dataset	ETTm1											
Baseline	0.381687	0.390652	0.39177	0.403024	0.398893	0.394252	0.281894	0.358602	0.2544	0.307104	0.252632	0.312604
Jitter	1.50E-05	-8.95E-03	1.39E-04	4.12E-04	-4.64E-03	-4.60E-05	1.40E-05	1.60E-05	6.46E-03	3.53E-03	-1.81E-04	-2.21E-04
Hflip	-8.20E-05	-2.56E-04	3.84E-03	2.34E-04	-1.65E-02	-2.11E-03	2.81E-03	3.15E-03	-2.67E-03	1.20E-03	-4.76E-03	-3.79E-03
Vflip	5.25E-04	1.00E-03	-2.07E-03	-4.10E-03	-6.76E-03	3.11E-03	-5.25E-03	-2.63E-03	-2.69E-03	1.28E-03	5.62E-02	-3.79E-03
Scaling	3.79E-04	7.50E-04	-2.01E-03	-1.02E-03	-1.80E-02	5.00E-05	2.25E-03	2.33E-03	7.54E-03	4.58E-03	-4.20E-05	5.30E-05
Win.warp	7.35E-02	4.84E-02	4.79E-02	3.62E-02	4.49E-02	3.92E-02	3.46E-03	4.96E-03	3.92E-03	5.27E-03	2.42E-03	3.48E-03
Win.slide	8.62E-02	5.08E-02	8.04E-02	4.75E-02	3.45E-02	3.92E-02	1.50E-03	2.18E-03	1.33E-02	1.04E-02	4.73E-03	4.28E-03
Permu	4.45E-02	3.36E-02	7.88E-03	1.13E-02	3.45E-02	1.95E-02	5.75E-03	7.47E-03	1.33E-02	2.23E-03	1.93E-03	2.91E-03
Mixup	1.54E-04	4.21E-04	-2.86E-03	-2.19E-03	-1.90E-02	-2.12E-03	-2.09E-03	-1.48E-03	-1.14E-03	-9.32E-04	-1.56E-03	-1.10E-03
TS.IA	4.96E-03	4.27E-03	-8.28E-03	-2.50E-03	-1.99E-02	-1.15E-03	-2.18E-02	-2.18E-02	-4.66E-03	-3.04E-03	-2.82E-03	-3.71E-03
iT.IA	5.44E-03	4.72E-03	-7.23E-03	-3.52E-03	-2.12E-02	-1.20E-03	2.06E-03	1.87E-03	-4.39E-03	-2.77E-03	-2.74E-03	-3.71E-03
TS.IMA	5.18E-03	4.27E-03	-7.62E-03	-4.76E-03	-2.17E-02	-3.63E-03	-1.49E-02	-1.61E-02	-5.80E-03	-2.50E-03	-4.03E-03	-4.44E-03
iT.IMA	6.27E-03	5.27E-03	-1.22E-02	-9.31E-03	-2.36E-02	-4.22E-03	-6.69E-03	-5.99E-03	-7.18E-03	-4.58E-03	-4.34E-03	-5.13E-03
Dataset	Illness											
Baseline	4.003106	1.441318	1.998755	0.885458	1.807093	0.870089	0.168927	0.305395	0.219712	0.340417	0.180669	0.303503
Jitter	0.00E+00	0.00E+00	-1.90E-05	-3.00E-06	0.00E+00	0.00E+00	0.00E+00	0.00E+00	-6.93E-04	-3.37E-04	0.00E+00	0.00E+00
Hflip	0.00E+00	0.00E+00	3.30E-05	7.00E-06	0.00E+00	0.00E+00	0.00E+00	0.00E+00	-3.72E-04	-2.52E-04	0.00E+00	0.00E+00
Vflip	0.00E+00	0.00E+00	-8.00E-06	3.00E-06	0.00E+00	0.00E+00	0.00E+00	0.00E+00	-1.44E-03	-1.29E-03	0.00E+00	0.00E+00
Scaling	0.00E+00	0.00E+00	1.10E-05	3.00E-06	0.00E+00	0.00E+00	0.00E+00	0.00E+00	-1.14E-03	-5.71E-04	0.00E+00	0.00E+00
Win.warp	0.00E+00	0.00E+00	1.90E-05	2.00E-06	0.00E+00	0.00E+00	0.00E+00	0.00E+00	-1.90E-03	-7.10E-04	0.00E+00	0.00E+00
Win.slide	0.00E+00	0.00E+00	1.30E-05	1.00E-06	0.00E+00	0.00E+00	0.00E+00	0.00E+00	-8.52E-04	-6.02E-04	0.00E+00	0.00E+00
Permu	0.00E+00	0.00E+00	-2.30E-05	-2.00E-06	0.00E+00	0.00E+00	0.00E+00	0.00E+00	-3.70E-05	4.92E-04	0.00E+00	0.00E+00
Mixup	2.45E-02	4.30E-03	-1.06E-01	-2.64E-02	3.93E-02	3.99E-03	7.51E-03	8.34E-03	-8.73E-03	-9.11E-03	-1.08E-03	-1.46E-03
TS.IA	-9.15E-03	-7.45E-04	-7.46E-02	-2.10E-02	3.21E-02	6.48E-03	4.18E-03	6.24E-03	-1.65E-02	-1.59E-02	-1.90E-03	-2.32E-03
iT.IA	-9.85E-03	-9.53E-04	-7.03E-02	-2.40E-02	4.09E-02	3.00E-03	7.16E-03	8.38E-03	4.83E-03	1.75E-03	-1.46E-03	-1.51E-03
TS.IMA	-1.92E-03	2.27E-04	-2.67E-03	-1.23E-02	1.04E-01	2.12E-02	3.56E-03	5.04E-03	-8.48E-03	-9.61E-03	-1.84E-03	-2.12E-03
iT.IMA	1.60E-03	1.74E-03	-1.43E-01	-2.54E-02	7.62E-02	1.13E-02	7.68E-03	1.00E-02	-1.54E-02	-1.48E-02	-1.49E-03	-1.89E-03
Dataset	Exchange Rate											
Baseline	0.00E+00	0.00E+00	0.00E+00	0.00E+00	0.00E+00	0.00E+00	0.00E+00	0.00E+00	0.00E+00	0.00E+00	0.00E+00	0.00E+00
Jitter	0.00E+00	0.00E+00	0.00E+00	0.00E+00	0.00E+00	0.00E+00	0.00E+00	0.00E+00	-6.93E-04	-3.37E-04	0.00E+00	0.00E+00
Hflip	0.00E+00	0.00E+00	0.00E+00	0.00E+00	0.00E+00	0.00E+00	0.00E+00	0.00E+00	-3.72E-04	-2.52E-04	0.00E+00	0.00E+00
Vflip	0.00E+00	0.00E+00	-8.00E-06	3.00E-06	0.00E+00	0.00E+00	0.00E+00	0.00E+00	-1.44E-03	-1.29E-03	0.00E+00	0.00E+00
Scaling	0.00E+00	0.00E+00	1.10E-05	3.00E-06	0.00E+00	0.00E+00	0.00E+00	0.00E+00	-1.14E-03	-5.71E-04	0.00E+00	0.00E+00
Win.warp	0.00E+00	0.00E+00	1.90E-05	2.00E-06	0.00E+00	0.00E+00	0.00E+00	0.00E+00	-1.90E-03	-7.10E-04	0.00E+00	0.00E+00
Win.slide	0.00E+00	0.00E+00	1.30E-05	1.00E-06	0.00E+00	0.00E+00	0.00E+00	0.00E+00	-8.52E-04	-6.02E-04	0.00E+00	0.00E+00
Permu	0.00E+00	0.00E+00	-2.30E-05	-2.00E-06	0.00E+00	0.00E+00	0.00E+00	0.00E+00	-3.70E-05	4.92E-04	0.00E+00	0.00E+00
Mixup	2.45E-02	4.30E-03	-1.06E-01	-2.64E-02	3.93E-02	3.99E-03	7.51E-03	8.34E-03	-8.73E-03	-9.11E-03	-1.08E-03	-1.46E-03
TS.IA	-9.15E-03	-7.45E-04	-7.46E-02	-2.10E-02	3.21E-02	6.48E-03	4.18E-03	6.24E-03	-1.65E-02	-1.59E-02	-1.90E-03	-2.32E-03
iT.IA	-9.85E-03	-9.53E-04	-7.03E-02	-2.40E-02	4.09E-02	3.00E-03	7.16E-03	8.38E-03	4.83E-03	1.75E-03	-1.46E-03	-1.51E-03
TS.IMA	-1.92E-03	2.27E-04	-2.67E-03	-1.23E-02	1.04E-01	2.12E-02	3.56E-03	5.04E-03	-8.48E-03	-9.61E-03	-1.84E-03	-2.12E-03
iT.IMA	1.60E-03	1.74E-03	-1.43E-01	-2.54E-02	7.62E-02	1.13E-02	7.68E-03	1.00E-02	-1.54E-02	-1.48E-02	-1.49E-03	-1.89E-03

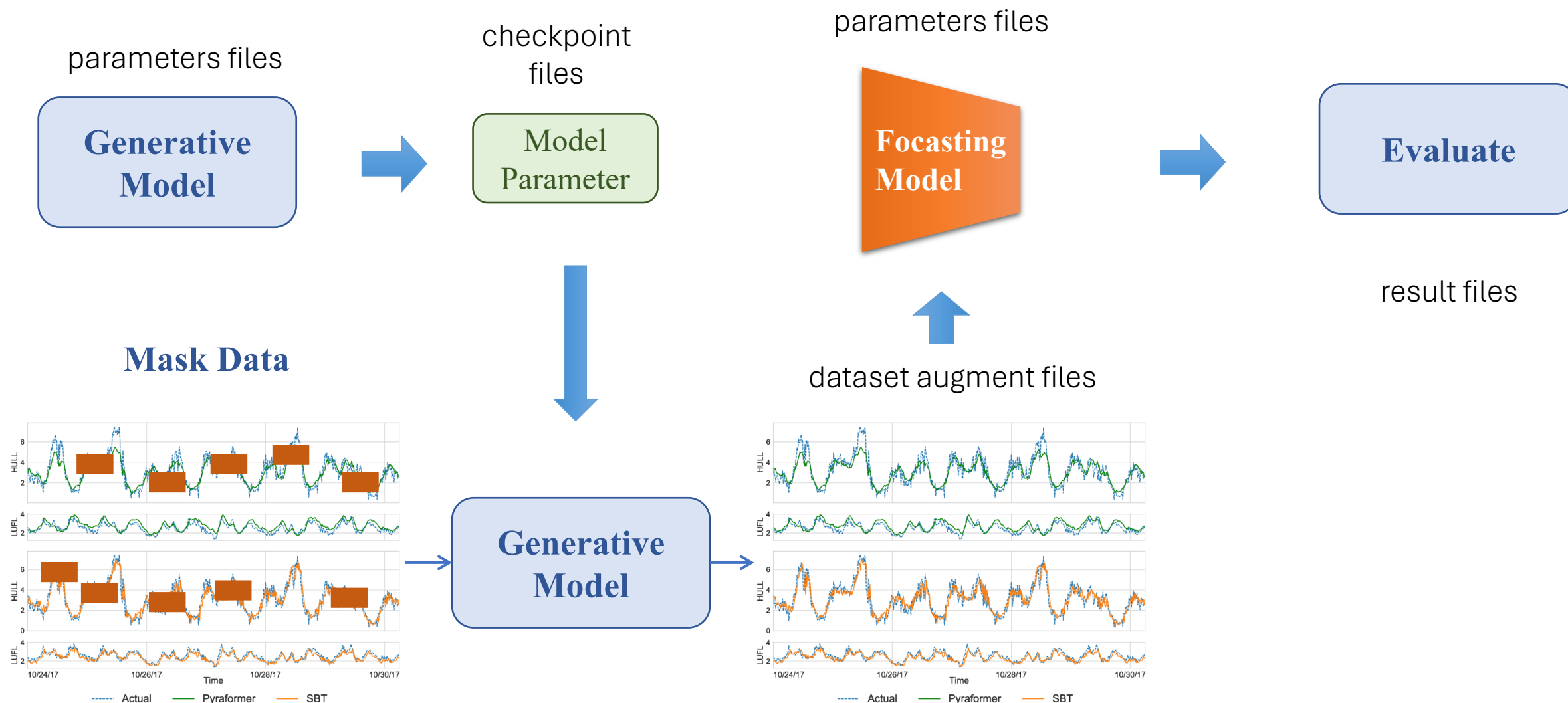
Theory Perspective

❖ My problem on Data Version



Theory Perspective

❖ My problem on Data Version



Theory Perspective

❖ 3 Big Problems

- **Team work**
- Tons of data file version
- Data file and code are not connected



My teammate

- Pull new code
- Download new dataset, params files from Google Drive
- Import those files into source code

Dataset

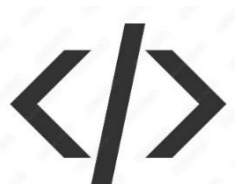


Params



I have new aug method, try on new dataset, can you run next step

Code



GitHub



Theory Perspective

❖ 3 Big Problems

- Team work
- **Tons of data file version**
- Data file and code are not connected



- model.pkl
- model_mix05_mask25.pkl
- cnn_impuse_mix05_mask075.pkl
- cnn_ettml_mix1_mask025.pkl

Dataset



Params



I have tried new aug
method (1,2,3) on
many new dataset
(a,b,c)

Code



GitHub



Theory Perspective

❖ 3 Big Problems

- Team work
- Tons of data file version
- **Data file and code are not connected**



Hmmm ... Is this data file use to train CNN?

Is this parameter files are output of tranformer impusion?

Where the params file of focasting model?

Dataset



Params



I have tried new aug method (1,2,3) on many new dataset (a,b,c)

Code

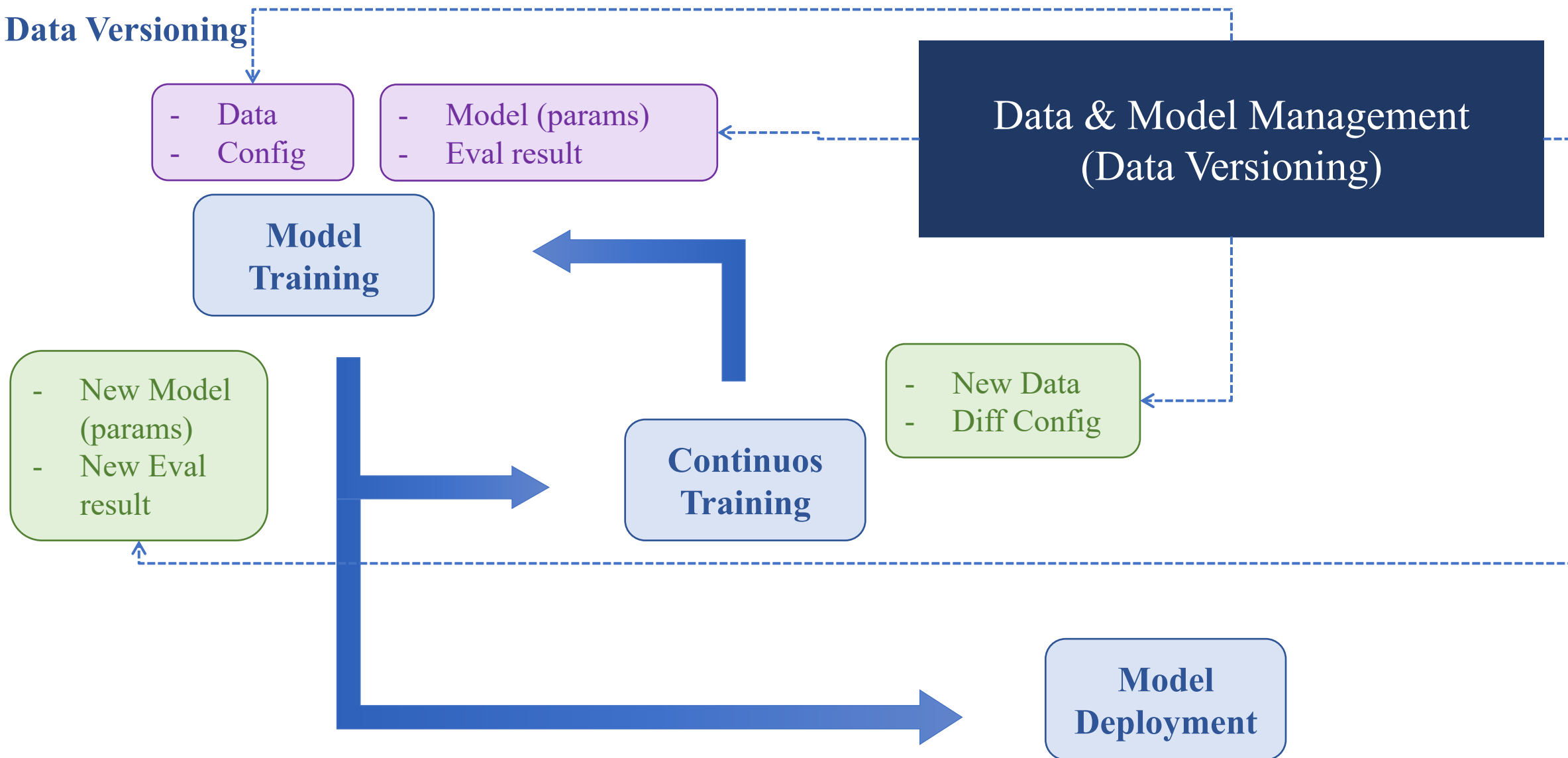


GitHub



Theory Perspective

❖ Data Versioning



Theory Perspective

❖ Data Version Control (DVC)

	Open Source	Data Format Agnostic	Cloud/Storage Agnostic* (Supports most common cloud and storage types)	Simple to Use	Easy Support for BIG Data
	✓ (Apache 2.0)	✓	✓	✓	✗
	✓ (Apache 2.0)	✗	✓	✗	✓
	✓ (MIT)	✓	✗	✓	✗
	✓⚠ (Non-standard license)	✓	✓	✗	✓
	✓ (Apache 2.0)	✗	✗	✗	✗
	✓ (Apache 2.0)	✓	✗	✗	✓

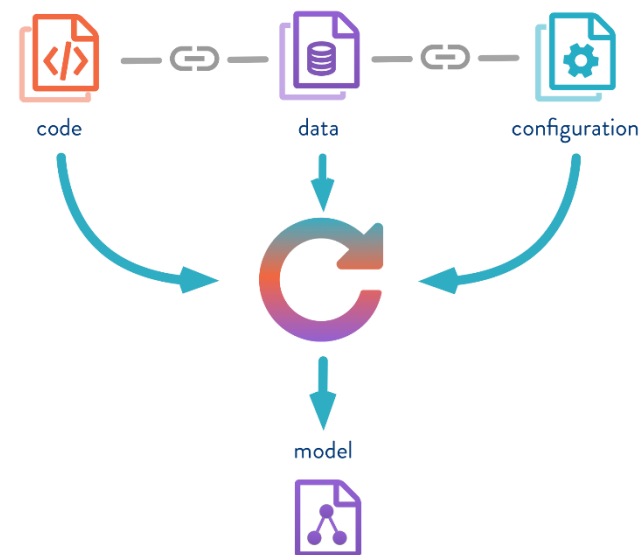

DAGsHub

Theory Perspective

❖ Data Version Control (DVC)



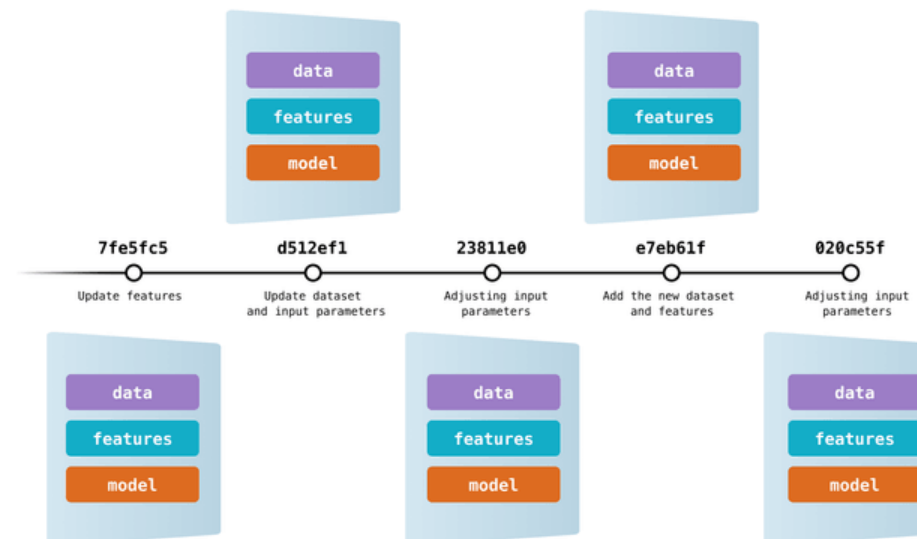
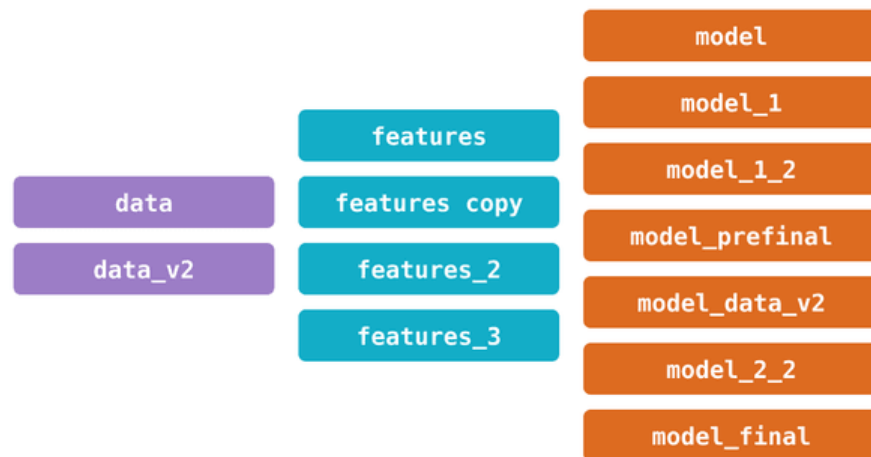
- Open Source tool Data Version Control to manage ML Projects: <https://dvc.org/>
- DVC is a data science platform on top of open source stack
- Easy download: `pip install dvc`
- Easy command like Git: `dvc add`, `dvc run`, `dvc repo`, `dvc remote`



Theory Perspective

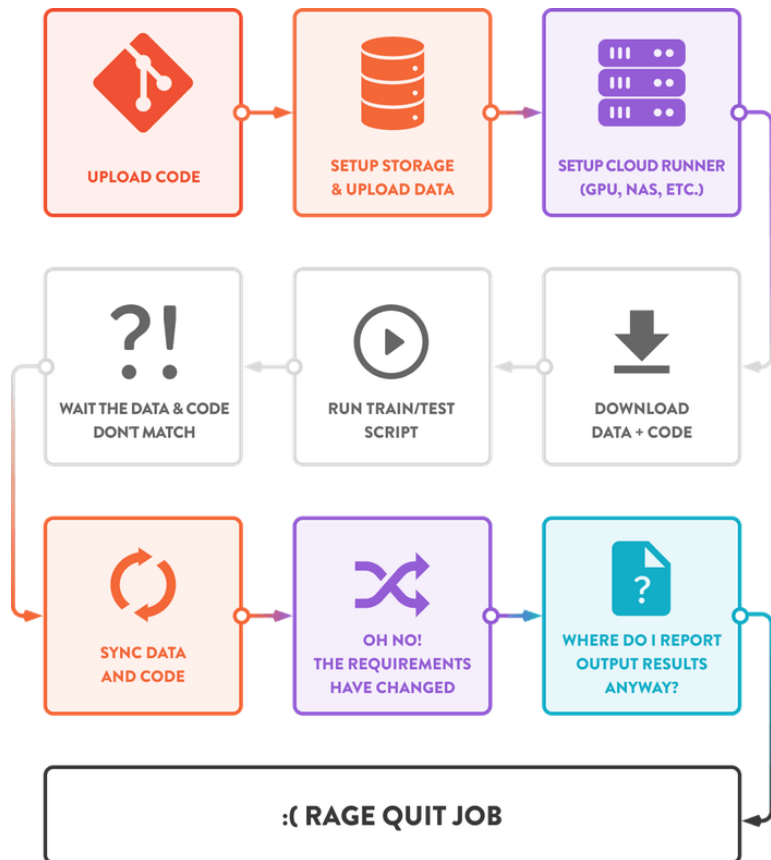
❖ Data Version Control (DVC)

➤ Experiment as commit/branch: Code + Data files

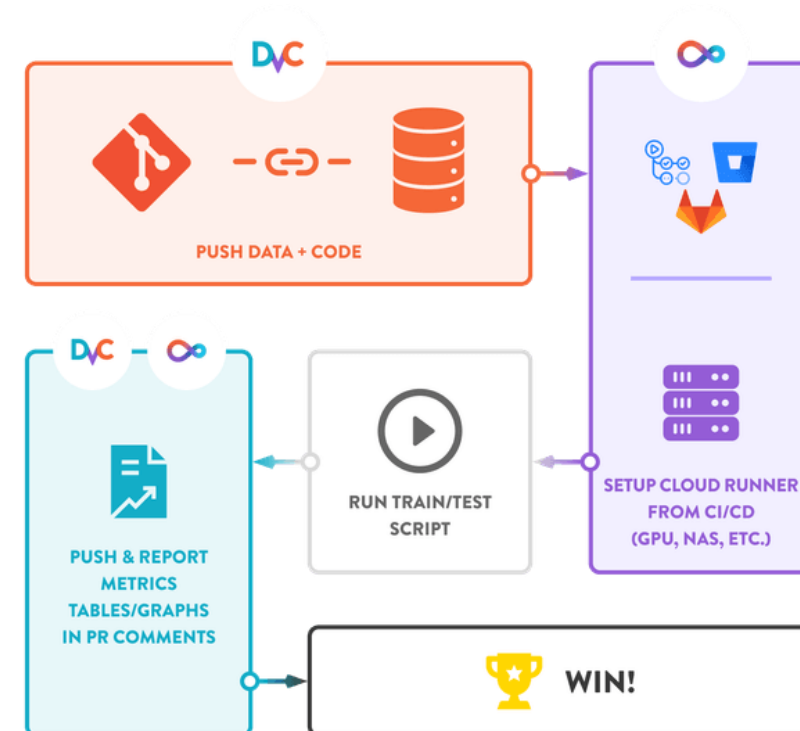


Theory Perspective

❖ Data Version Control (DVC)

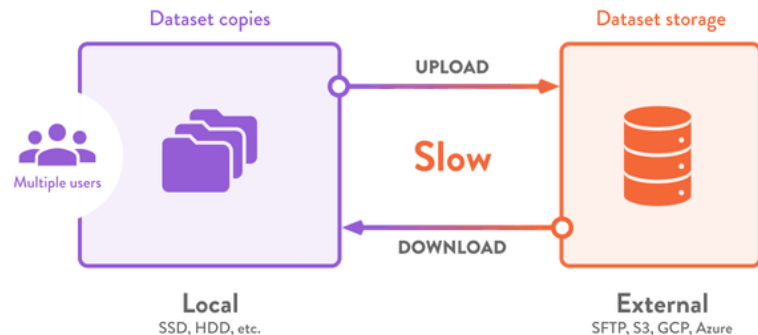


➤ Machine learning pipeline

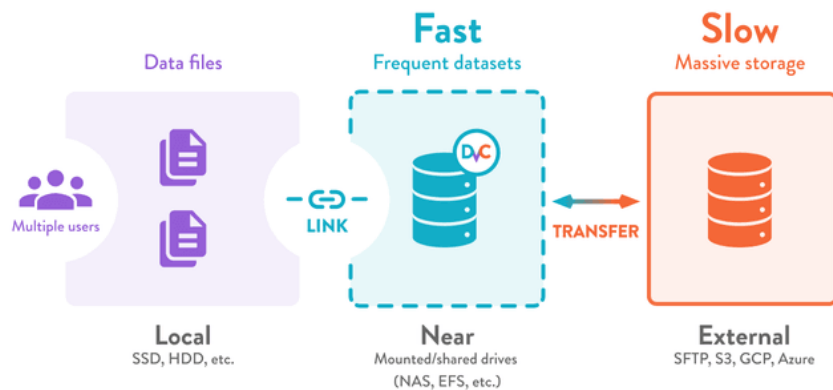


Theory Perspective

❖ Data Version Control (DVC)

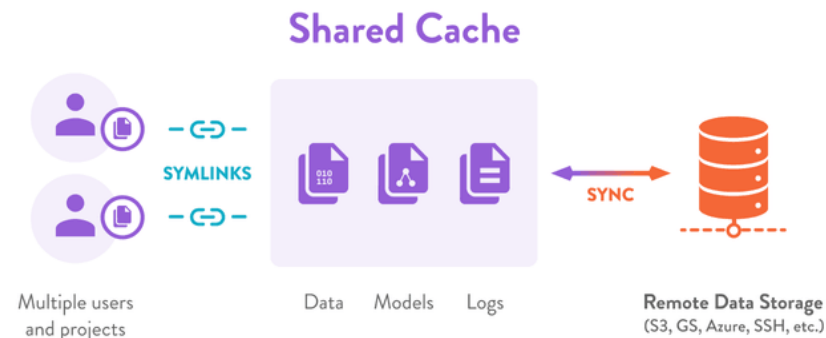


Local Cache



Middleware Data Storage

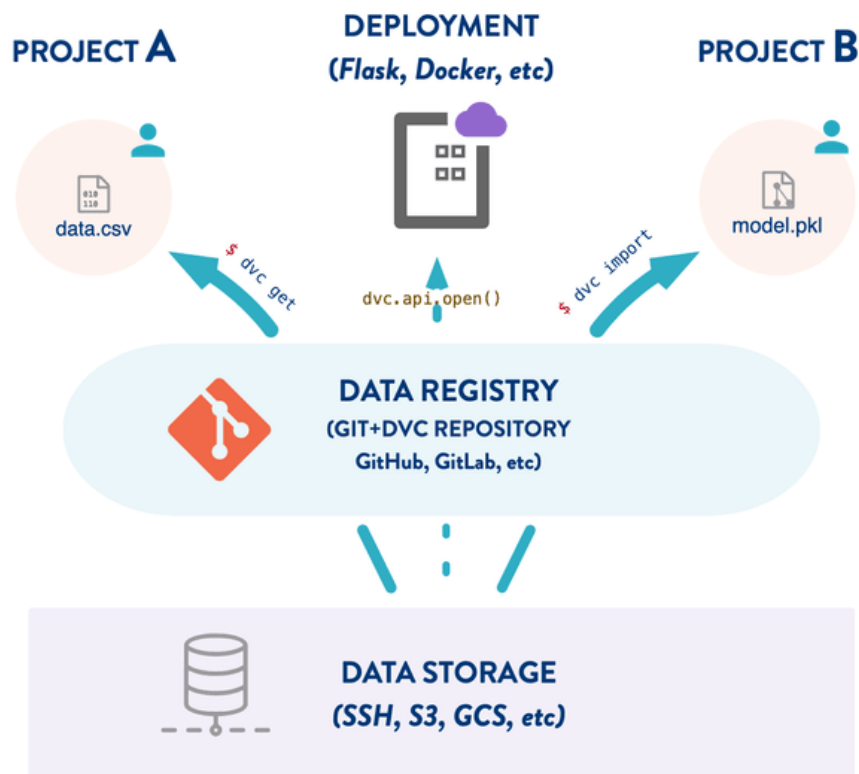
- Large data files:
 - Local cache
 - Optimize for 1-100gb file size
 - Data remote storage: S3, GCP, SSH



Example

Theory Perspective

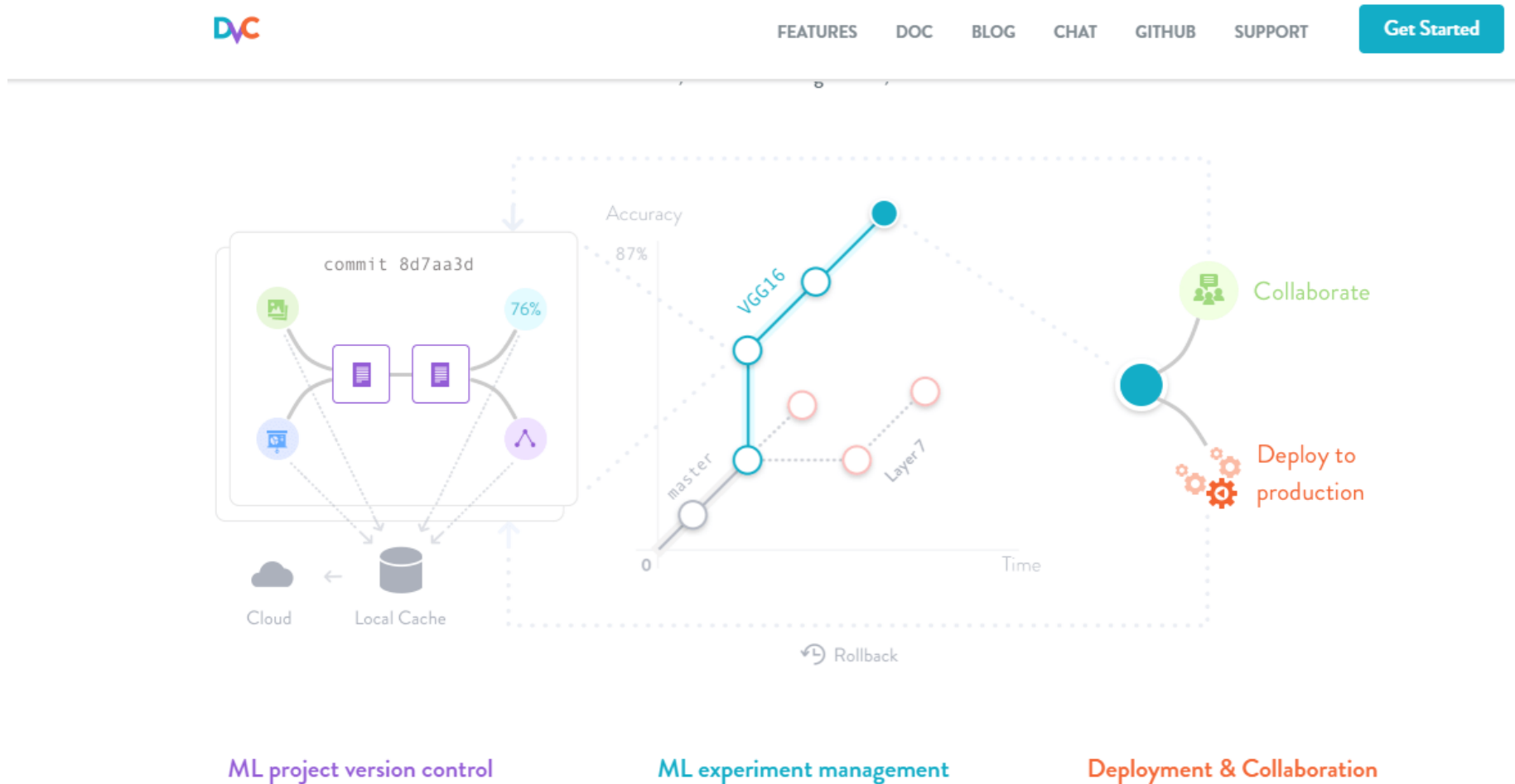
❖ Data Version Control (DVC)



- Large data files:
 - Local cache
 - Optimize for 1-100gb file size
 - Data remote storage: S3, GCP, SSH

Theory Perspective

❖ Data Version Control (DVC)



Case Study Implementation



Case Study Implementation

❖ Step 1: Set Up Project

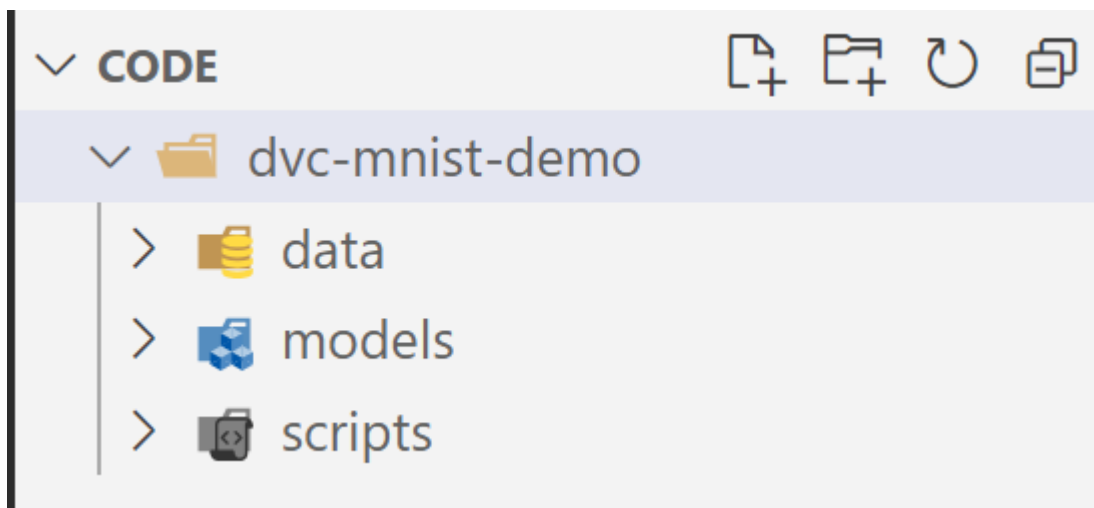
Create project directory

```
$ mkdir dvc-mnist-demo  
$ cd dvc-mnist-demo
```

Create basic folder structure

```
$ mkdir data/raw models scripts
```

```
$ mkdir data/raw  
$ mkdir data/processed  
$ mkdir models  
$ mkdir scripts
```





Case Study Implementation

❖ Step 1: Set Up Project

Create conda env or venv

```
$ conda create -n dvc_mnist python=3.11
```

```
$ conda activate dvc_mnist
```

Install Requirements

```
$ pip install -r requirements.txt
```

Creating download_v1.py file (Code in **Init project commit**)

Git repo commands

```
$ git init
```

```
$ git add .
```

```
$ git commit -m "Init project"
```



Case Study Implementation

❖ Step 1: Set Up Project

Git

master



Init Project

```
> 📁 data
> 📁 models
✓ 📁 scripts
  🐍 download_v1.py
  📄 requirements.txt
```



Case Study Implementation

❖ Step 2: Download Dataset and Set up DVC

Download dataset

```
$ python scripts/download_v1.py
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434  2s 0us/step
Version 1 - Full dataset: (60000, 28, 28)
```

✓	data	●
✓	processed	
✓	raw	
	x_test.npy	U
	x_train_v1.npy	U
	y_test.npy	U
	y_train_v1.npy	U

DVC set up and tracking files

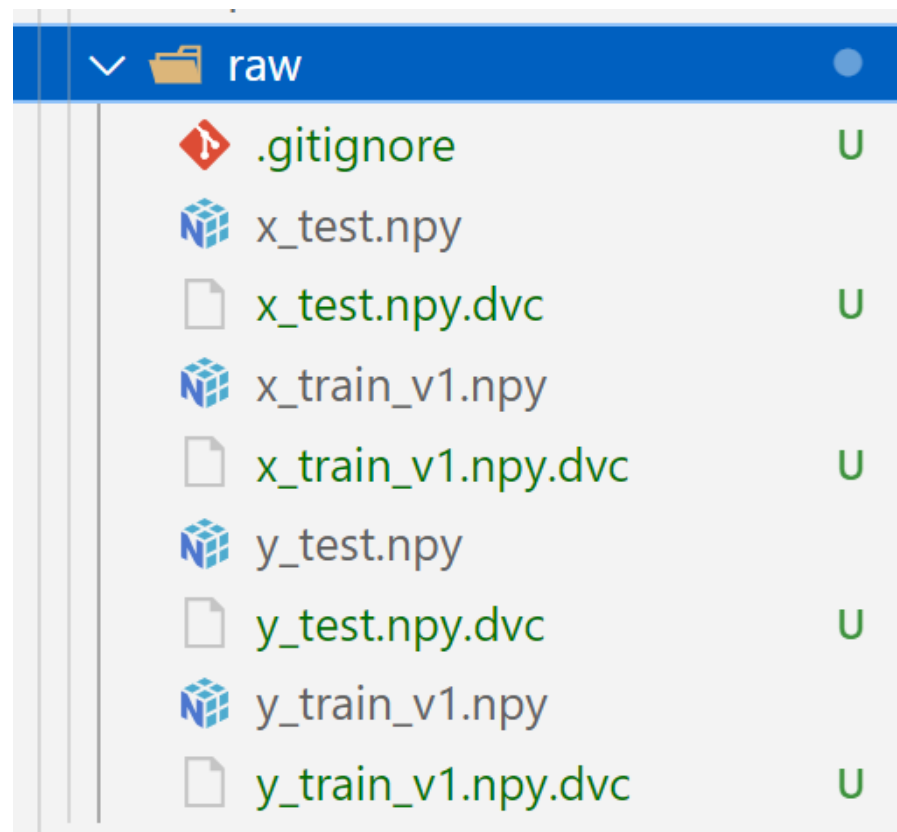
```
$ dvc init
```

```
$ dvc add data/raw/x_train_v1.npy data/raw/y_train_v1.npy data/raw/x_test.npy data/raw/y_test.npy
```

Case Study Implementation

❖ Step 2: Download Dataset and Set up DVC

```
$ dvc add data/raw/x_train_v1.npy data/raw/y_train_v1.npy data/raw/x_test.npy data/raw/y_test.npy
```





Case Study Implementation

❖ Step 3: Commit to Git

```
# Commit to git
```

```
$ git add data/raw/.gitignore data/raw/x_train_v1.npy.dvc data/raw/y_train_v1.npy.dvc data/raw/x_test.npy.dvc data/raw/y_test.npy.dvc
```

```
$ git commit -m "Version 1: Full MNIST dataset"
```

```
commit d682776577af4a4929a9ea8b294ec68577b957ec (HEAD -> master)
```

```
Author: dangnha <nhand.21it@vku.udn.vn>
```

```
Date:   Wed Oct 1 15:08:53 2025 +0700
```

```
Version 1: Full MNIST dataset
```

```
commit 322e02664d658b7f302addfeb79f440a66e89417
```

```
Author: dangnha <nhand.21it@vku.udn.vn>
```

```
Date:   Wed Oct 1 14:54:22 2025 +0700
```

```
Init project
```



❖ Step 3: Commit to Git

Version 1: Full MNIST dataset

DVC

master



Git

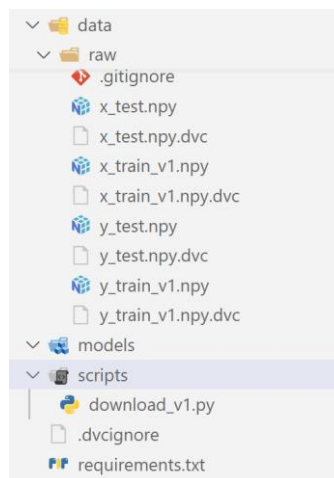
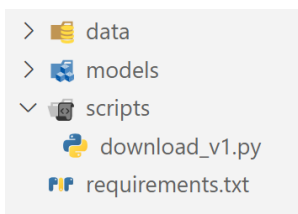
master



Init Project



Version 1: Full MNIST dataset





Case Study Implementation

❖ Step 4: Training

Create training script (Code in commit “**Model v1: Full dataset accuracy**”)

Create symbolic link for data train (like shortcut)

```
$ cd data/raw
$ mklink x_train.npy x_train_v1.npy
$ mklink y_train.npy y_train_v1.npy
$ cd ../..
```

Training

```
$ python scripts/train.py
```

```
Loading data...
Training on 60000 samples...
Accuracy: 0.9319
Model and metrics saved!
```

models		
	metrics.json	U
	model.npy	U



Case Study Implementation

❖ Step 4: Training

Add dvc track

```
$ dvc add models/model.npy  
$ dvc add models/metrics.json
```

Git commit

```
$ git add .  
$ git commit -m "Model v1: Full dataset accuracy"
```

```
commit 9511885a667abc946c972fbdb433afb653304b1f (HEAD -> master)  
Author: dangnha <nhand.21it@vku.udn.vn>  
Date:   Wed Oct 1 15:55:06 2025 +0700
```

Model v1: Full dataset accuracy

```
commit 7cc0197861e5f86402819d8313d9ba688a1a93d3  
Author: dangnha <nhand.21it@vku.udn.vn>  
Date:   Wed Oct 1 15:45:47 2025 +0700
```

Version 1: Full MNIST dataset

```
commit b8d7edd8c3954ab7e50f19d283402f782c4a3c0d  
Author: dangnha <nhand.21it@vku.udn.vn>  
Date:   Wed Oct 1 15:40:20 2025 +0700
```

Init Project

Case Study Implementation

❖ Step 4: Training

Version 1: Full MNIST dataset

DVC

master



Git

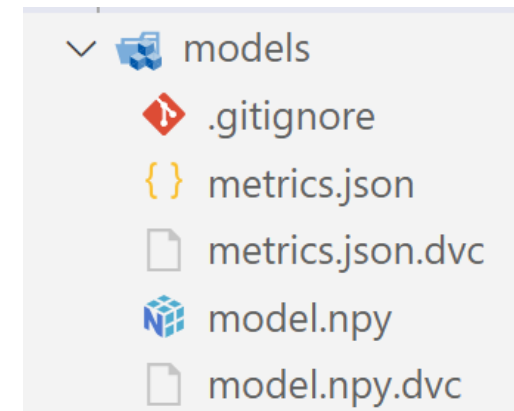
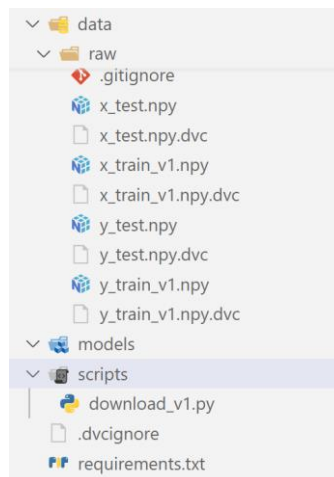
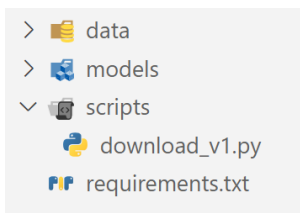
master



Init Project

Version 1: Full MNIST dataset

Model v1: Full dataset
accuracy





❖ Step 4.1: Push data to Local Storage

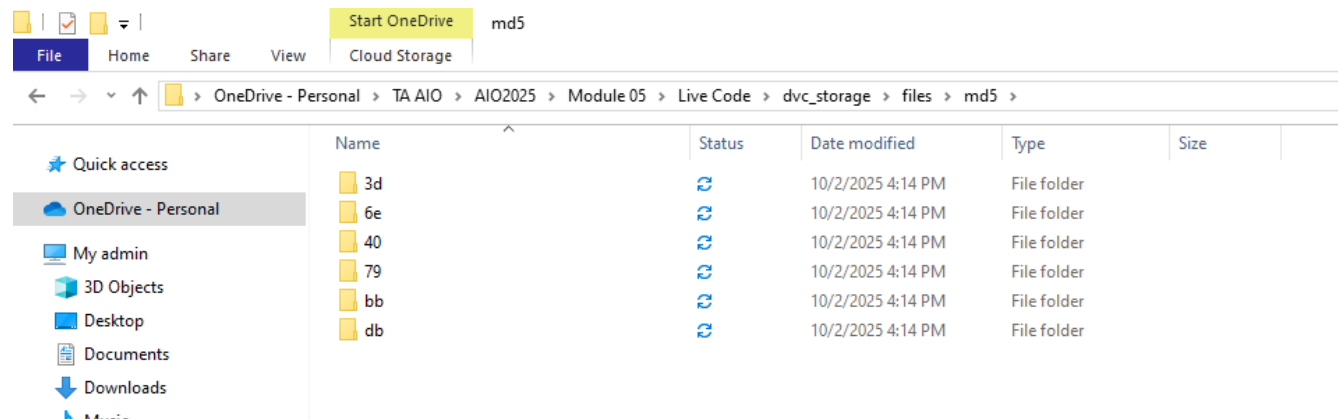
Create Local Storage

```
$ mkdir dvc_storage
```

```
$ dvc remote add -d localremote ./dvc_storage
```

Push data to storage

```
$ dvc push
```





Case Study Implementation

❖ Step 5: Download Dataset v2

```
# Create scripts/download_v2.py (Code in commit "Dataset V2")
```

```
# Download
```

```
$ python scripts/download_v2.py
```

```
Creating version 2 (small dataset)...
```

```
V2 - Training data: (1000, 28, 28) (1000 samples)
```

```
# Add into dvc
```

```
$ dvc add data/raw/x_train_v2.npy
```

```
$ dvc add data/raw/y_train_v2.npy
```

```
# Add git -> commit
```

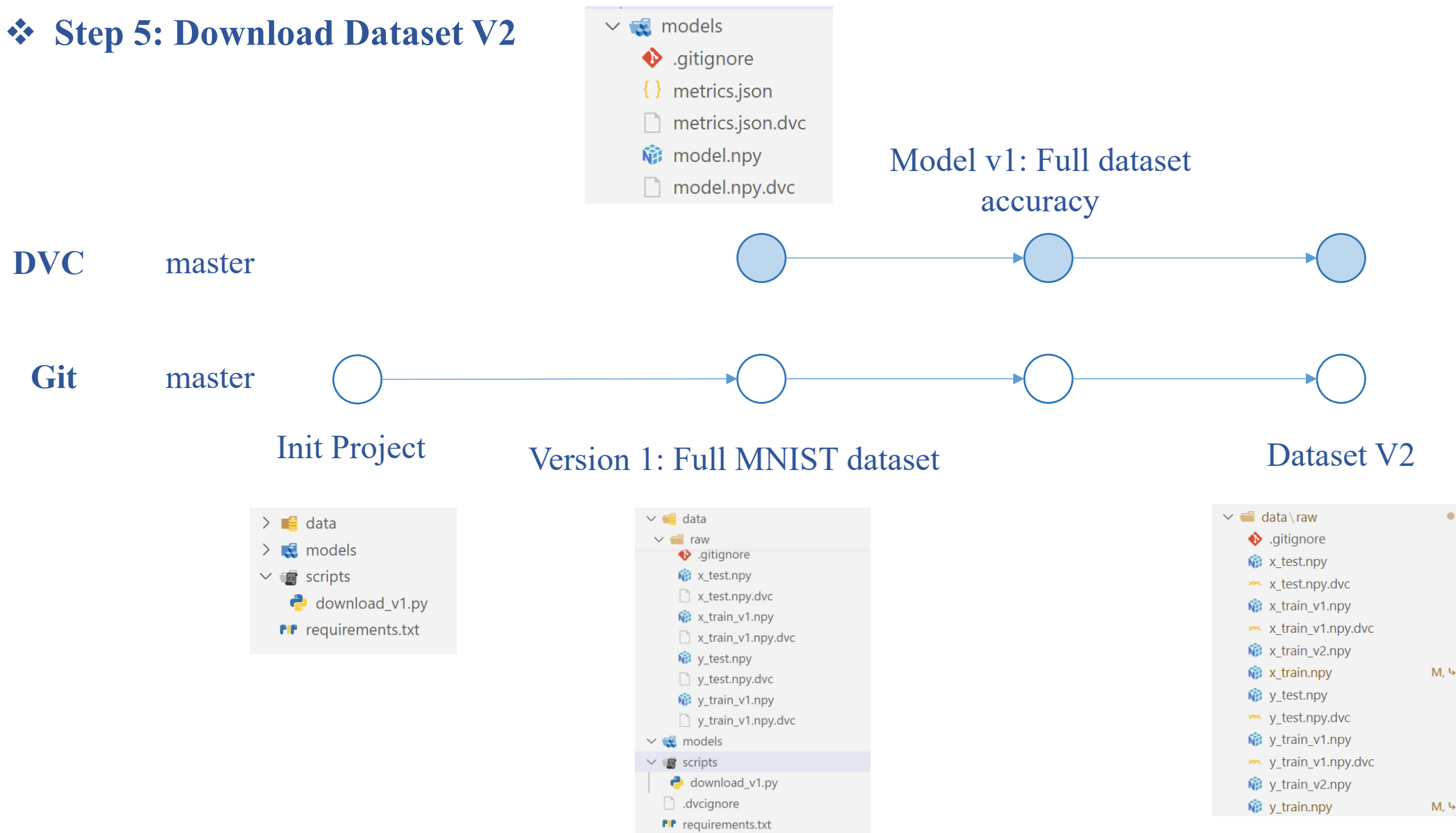
```
$ git add .
```

```
$ git commit -m "Dataset V2"
```



Case Study Implementation

❖ Step 5: Download Dataset V2





Case Study Implementation

❖ Step 6: Training Ver 2

Create training script (Code in commit “Model v2: Small dataset accuracy”)

Create symbolic link for data train (like shortcut)

```
$ cd data/raw  
$ del x_train.npy y_train.npy  
$ mklink x_train.npy x_train_v2.npy  
$ mklink y_train.npy y_train_v2.npy  
$ cd ../..
```

Training

```
$ python scripts/train.py
```

```
Training on 1000 samples...  
Accuracy: 0.8047  
Model and metrics saved!
```



Case Study Implementation

❖ Step 7: Add model ver 2 to DVC

Add dvc track

```
$ dvc add models/model.npy
```

```
$ dvc add models/metrics.json
```

Git commit

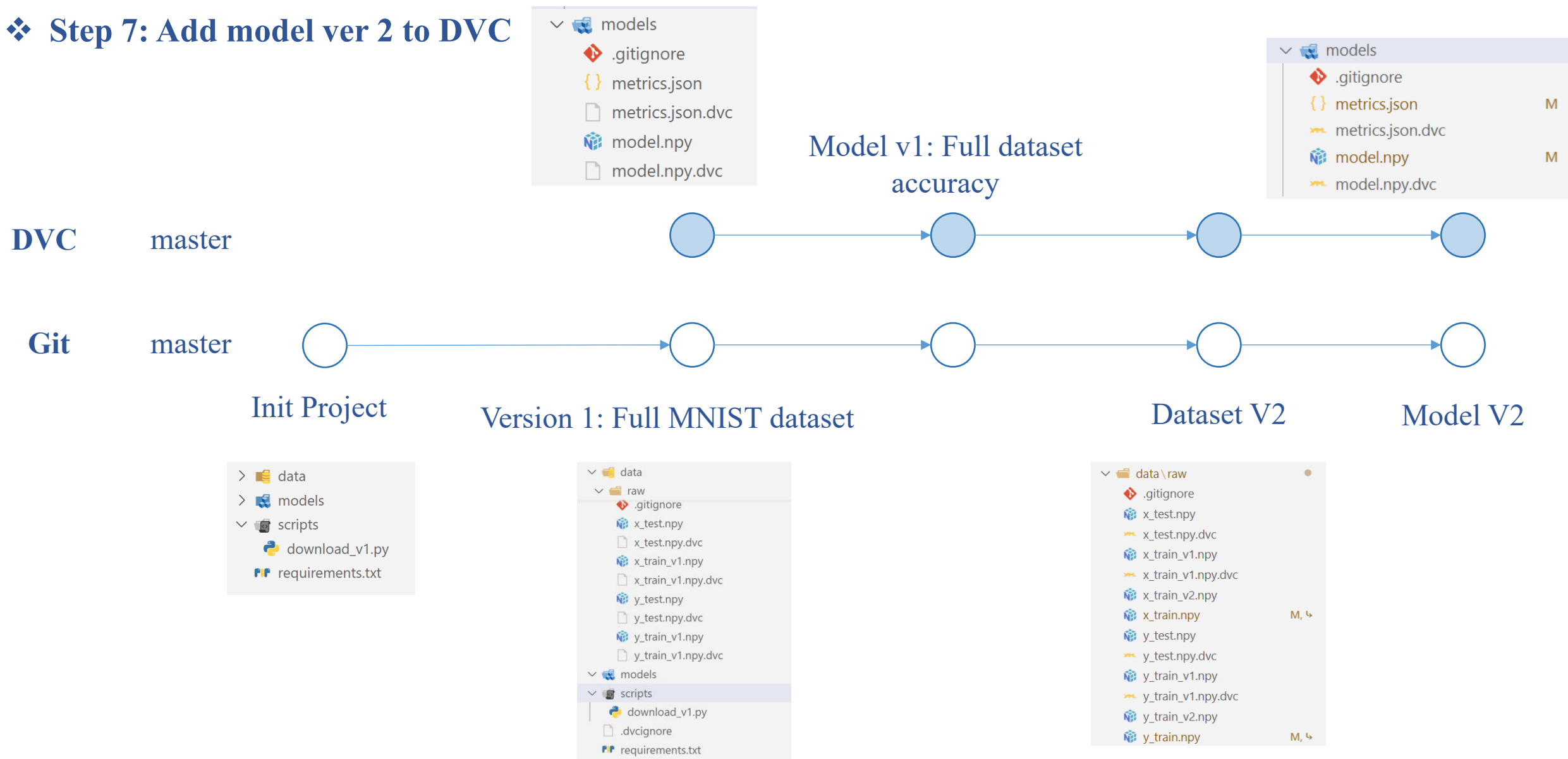
```
$ git add .
```

```
$ git commit -m "Model v2: Small dataset accuracy"
```



Case Study Implementation

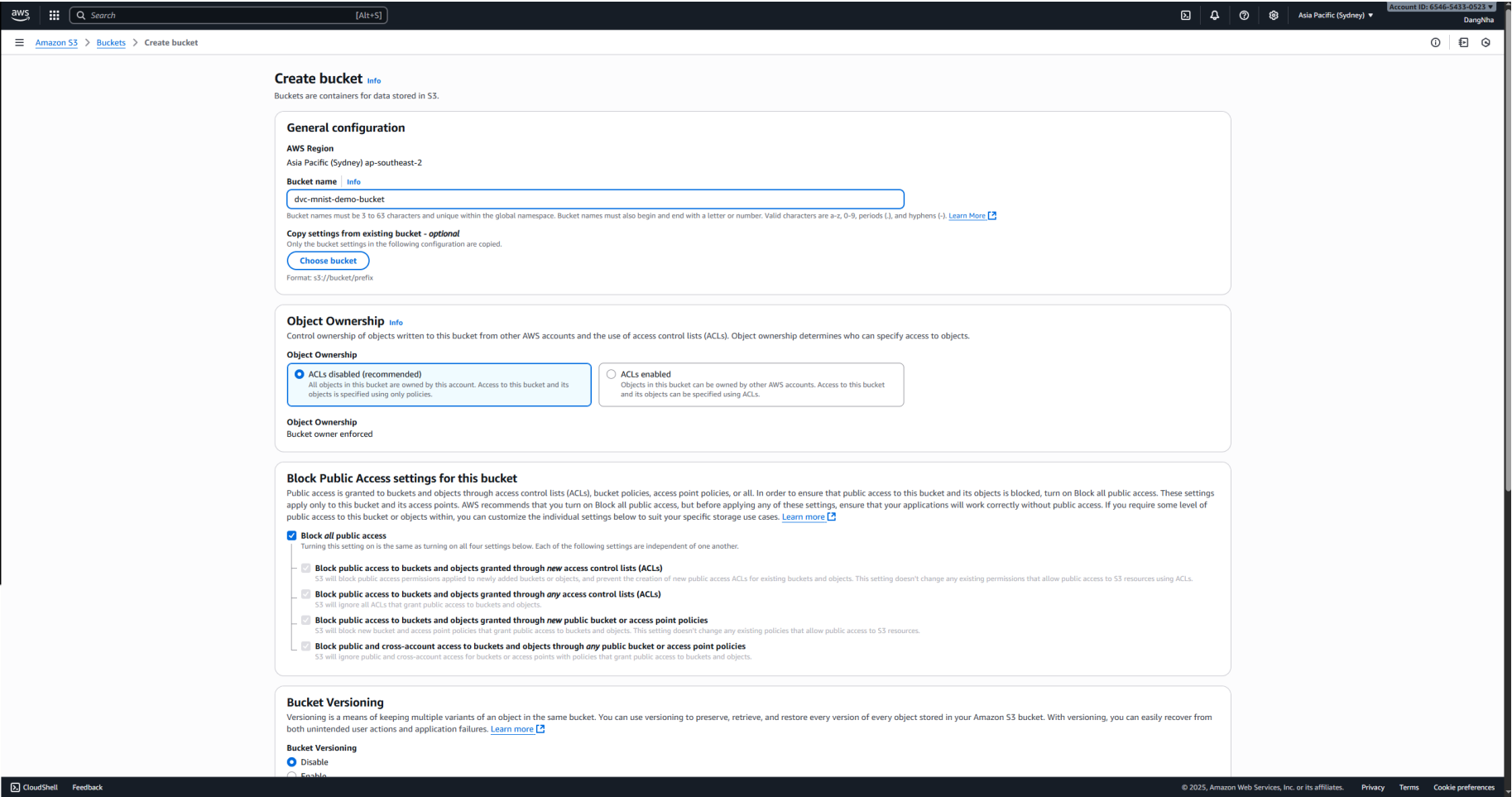
❖ Step 7: Add model ver 2 to DVC





Case Study Implementation

❖ Step 8: Set up DVC storage





Case Study Implementation

❖ Step 8: Set up DVC storage

Just try if you have free trial!!

Add remote storage

```
$ dvc remote add -d mys3 s3://dvc-mnist-demo-bucket/data  
$ pip install dvc-s3  
$ dvc push
```

dvc-mnist-demo-bucket [Info](#)

[Objects](#) | [Properties](#) | [Permissions](#) | [Metrics](#) | [Management](#) | [Access Points](#)

Objects (1) [Refresh](#) [Copy S3 URI](#) [Copy URL](#) [Download](#) [Open](#) [Delete](#) [Actions](#) [Create folder](#) [Upload](#)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

☐	Name	Type	Last modified	Size	Storage class
☐	data/	Folder	-	-	-



Case Study Implementation

❖ Step 8: Set up DVC storage

Add remote storage

```
$ dvc remote add -d mys3 s3://dvc-mnist-demo-bucket/data
```

```
(dvc_mnist) D:\OneDrive\TA AI0\AI02025\Module 05\Code\dvc-mnist-demo>dvc remote list  
mys3      s3://dvc-mnist-demo-bucket/data (default)
```

```
$ pip install dvc-s3
```

```
$ dvc push
```

dvc-mnist-demo-bucket [Info](#)

[Objects](#) | [Properties](#) | [Permissions](#) | [Metrics](#) | [Management](#) | [Access Points](#)

Objects (1)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

🔍 Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	data/	Folder	-	-	-



Case Study Implementation

❖ Step 9: Check Version Switching

Checkout Version 1

```
$ git checkout <commit_id> # Go to version 1 commit
```

```
$ dvc checkout
```

```
$ python -c "import numpy as np; print('V1 Data shape:', np.load('data/x_train.npy').shape)"
```



Case Study Implementation

❖ Step 10: Push and Clone

Push all code to GitHub

```
$ git remote add origin <your-repository>  
$ git push origin master
```

Clone code and add S3 remote

```
$ git clone <your-repository>  
$ dvc remote add -d mys3 s3://dvc-mnist-demo-bucket/data
```




Case Study Implementation

❖ Step 11: DVC Operations

Push all code to GitHub

```
$ dvc list .  
$ dvc status  
$ dvc pull
```

```
(dvc_mnist) D:\OneDrive\TA AI0\AI02025\Module 05\dvc-mnist-demo>dvc list .  
.dvcignore  
data  
models  
requirements.txt  
scripts
```

Basic DVC workflow

```
$ dvc add data/raw/file.npy      # Track data file  
$ dvc checkout                  # Switch data versions  
$ dvc push                      # Send to remote  
$ dvc pull                      # Get from remote  
$ dvc status                    # Check changes
```



Automate Pipelines with DVC



Automate Pipeline with DVC

❖ Configure File: dvc.yaml

```
stages:
  download_v1:
    cmd: python scripts/download_v1.py
    deps:
      - scripts/download_v1.py
    outs:
      - data/raw/x_train_v1.npy
      - data/raw/y_train_v1.npy
      - data/raw/x_test.npy
      - data/raw/y_test.npy

  download_v2:
    cmd: python scripts/download_v2.py
    deps:
      - scripts/download_v2.py
      - data/raw/x_train_v1.npy
      - data/raw/y_train_v1.npy
    outs:
      - data/raw/x_train_v2.npy
      - data/raw/y_train_v2.npy
```

```
train_v1:
  cmd: python scripts/train.py --version v1
  deps:
    - scripts/train.py
    - data/raw/x_train_v1.npy
    - data/raw/y_train_v1.npy
    - data/raw/x_test.npy
    - data/raw/y_test.npy
  outs:
    - models/model_v1.npy
  metrics:
    - models/metrics_v1.json:
        cache: false

train_v2:
  cmd: python scripts/train.py --version v2
  deps:
    - scripts/train.py
    - data/raw/x_train_v2.npy
    - data/raw/y_train_v2.npy
    - data/raw/x_test.npy
    - data/raw/y_test.npy
  outs:
    - models/model_v2.npy
  metrics:
    - models/metrics_v2.json:
        cache: false
```



Automate Pipeline with DVC

❖ DVC Reproducition

Run entire pipeline

```
$ dvc repro
```

Check pipeline status

```
$ dvc dag
```

Check metrics result

```
$ dvc metrics show
```

```
(dvc_mnist) D:\OneDrive\TA AIO\AI02025\Module 05\Live Code\dvc>dvc metrics show
```

Path	accuracy	dataset_size	dataset_version	model
models\metrics_v1.json	0.9319	60000	v1	model_v1
models\metrics_v2.json	0.8047	1000	v2	model_v2



Automate Pipeline with DVC

❖ Configure File: params.yaml

```
vars:
  - data_version: ["v1", "v2"]

stages:
  download_v1:
    cmd: python scripts/download_v1.py
    deps:
      - scripts/download_v1.py
    outs:
      - data/raw/x_train_v1.npy
      - data/raw/y_train_v1.npy
      - data/raw/x_test.npy
      - data/raw/y_test.npy

  download_v2:
    cmd: python scripts/download_v2.py
    deps:
      - scripts/download_v2.py
      - data/raw/x_train_v1.npy
      - data/raw/y_train_v1.npy
    outs:
      - data/raw/x_train_v2.npy
      - data/raw/y_train_v2.npy
```

```
train_${data_version}:
  foreach: ${data_version}
  do:
    cmd: python scripts/train.py --version ${item}
    deps:
      - scripts/train.py
      - data/raw/x_train_${item}.npy
      - data/raw/y_train_${item}.npy
      - data/raw/x_test.npy
      - data/raw/y_test.npy
    params:
      - model
    outs:
      - models/model_${item}.npy
    metrics:
      - models/metrics_${item}.json:
          cache: false
```



Automate Pipeline with DVC

❖ DVC Reproducition

Run entire pipeline

```
$ dvc repro
```

Check pipeline status

```
$ dvc dag
```

Check metrics result

```
$ dvc metrics show
```

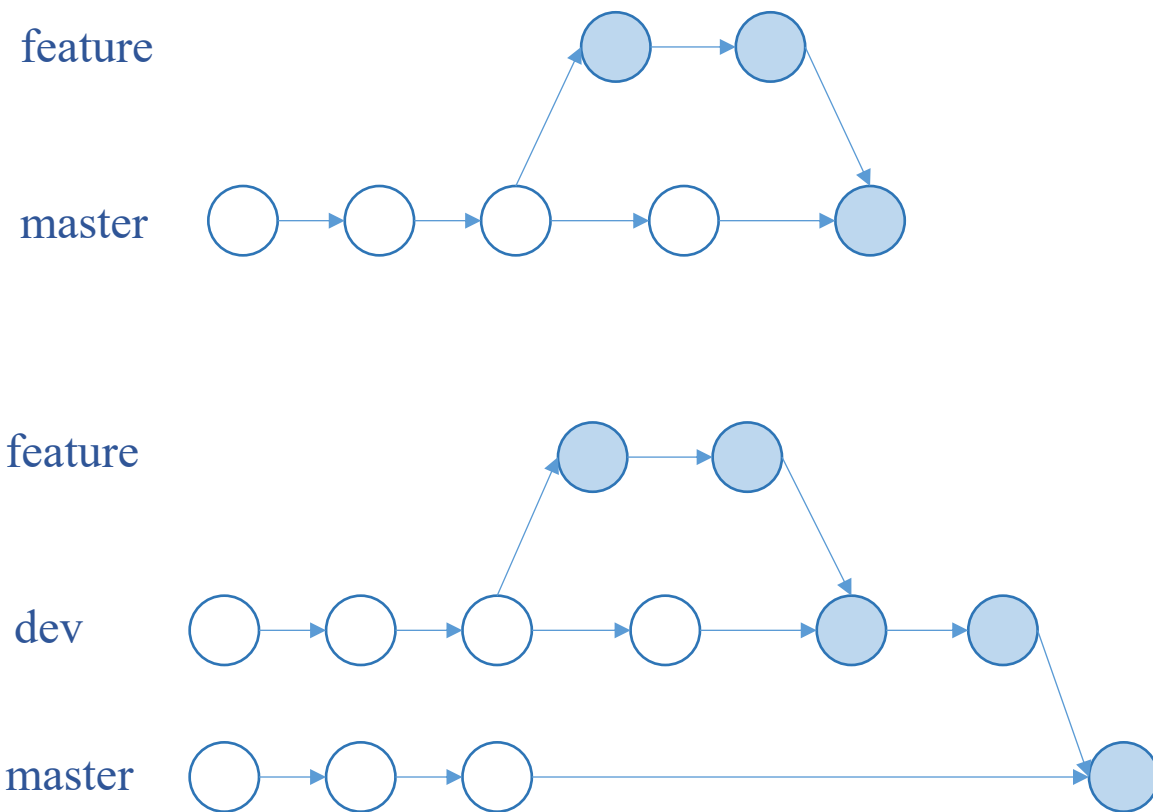
```
$ dvc params diff
```

```
$ dvc metrics diff
```

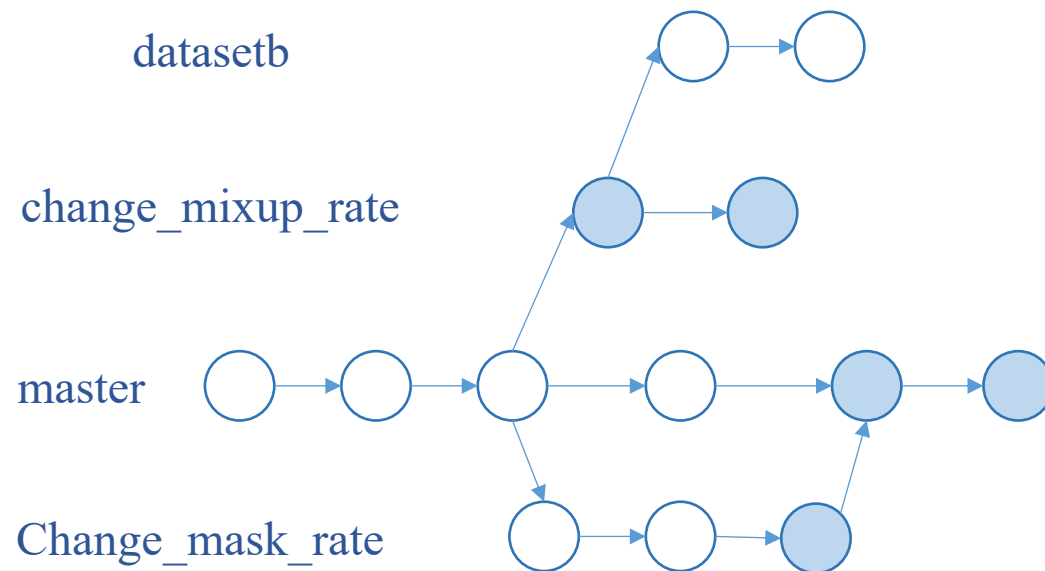
The Idea of Versioning

The Idea of Versioning

❖ Data Version Control (DVC)



Git Workflow



Data Workflow

Review

Theory Perspective

1. Recap AI, MLOPs
2. Data versioning?
3. ML – MLOPs pipeline
4. What is DVC?

Section 1

Case Study Implementation

1. Initialize Problem
2. DVC and Git setup
3. Step by step versioning data with DVC

Section 2

Automate Pipeline with DVC & The Idea of Versioning

1. Reproduction with DVC
2. Software & Data Experiment Workflow Diagram

Section 3

[Quiz&Feedbacks](#)

